

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO
COORDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

João Pessoa - PB
10/2026

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do Curso Superior de Tecnologia em Sistemas para Internet

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

Relatório de Estágio apresentado ao Coordenação do Curso Superior de Tecnologia em Sistemas para Internet da Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, como requisito parcial para a obtenção do título de Tecnólogo.

Orientador:	Juliana Dantas Ribeiro Viana de Medeiros
Supervisor:	Juliana Dantas Ribeiro Viana de Medeiros
Coordenador do Curso:	Cândido José Ramos do Egypto
Empresa:	Agência de Inovação (NEO-IFPB)
Período:	25/02/2026 à 19/06/2026

João Pessoa - PB

APROVAÇÃO

Esta folha deverá ser substituída pela cópia digitalizada da folha de aprovação fornecida.

Juliana Dantas Ribeiro Viana de Medeiros
Supervisor da Empresa

Cândido José Ramos do Egypto
Coordenador do CST de Sistemas para Internet

Prof. Dr. Juliana Dantas Ribeiro Viana de Medeiros
Professor Orientador

Luis Kilmer da Silva Bernardo
Estagiário

Altere este texto inserindo a dedicatória do seu trabalho.

AGRADECIMENTOS

Edite e coloque aqui os agradecimentos às pessoas e/ou instituições que contribuíram para a realização do trabalho.

É obrigatório o agradecimento às instituições de fomento à pesquisa que financiaram total ou parcialmente o trabalho, inclusive no que diz respeito à concessão de bolsas.

Eu denomino meu campo de Gestão do Conhecimento, mas você não pode gerenciar conhecimento. Ninguém pode. O que pode fazer - o que a empresa pode fazer - é gerenciar o ambiente que otimize o conhecimento. (PRUSAK, Laurence, 1997).

RESUMO

Silva Bernardo, Luis Kilmer. Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas. 10/2026. 54 f. Relatório de Estágio – Coordenação do Curso Superior de Tecnologia em Sistemas para Internet, Instituto Federal de Educação, Ciência e Tecnologia da Paraíba. João Pessoa - PB, 10/2026.

Este relatório descreve o desenvolvimento de um módulo de solicitações para o sistema Banco de Especialistas, com foco na formalização e no rastreio de solicitações de alteração de categoria e pontuação dos especialistas. A solução foi implementada com Python, Flask, PostgreSQL, SQLAlchemy e Marshmallow no back-end, além de integração com a arquitetura existente e testes automatizados com Pytest. O módulo permitiu registrar a intenção do pedido, controlar o fluxo de revisão, aplicar regras de negócio e manter histórico das decisões por meio de snapshot e timeline. Como resultado, o sistema passou a centralizar e documentar as alterações cadastrais de forma consistente, reduzindo a dependência de intervenções manuais e aumentando a rastreabilidade e a segurança do processo.

Palavras-chave: Desenvolvimento full-stack. Banco de Especialistas. Solicitações. Rastreabilidade. Testes automatizados.

LISTA DE FIGURAS

Figura 1 – Fluxo de solicitações manuais de ajuste cadastral e de categoria antes do desenvolvimento do módulo. 21

LISTA DE QUADROS

Quadro 1 – Requisitos funcionais do módulo 23
Quadro 2 – Requisitos não funcionais do módulo 24
Quadro 3 – Endpoints do módulo de solicitações e revisão 25
Quadro 4 – Modelo de dados do módulo de solicitações 27
Quadro 5 – Modelo da entidade SpecialistSnapshot 32
Quadro 6 – Distribuição dos testes por arquivo 41

LISTA DE TABELAS

Tabela 1 – Códigos HTTP e suas aplicações no módulo 6
Tabela 2 – Papéis e permissões no módulo de solicitações. 15
Tabela 3 – Volume de chamados por tipo de inscrição 21
Tabela 4 – Distribuição dos chamados por categoria de problema 21

LISTA DE ALGORITMOS

Algoritmo 1 – Padrão de implementação observado em controladores existentes. . . . 18
Algoritmo 2 – Estrutura do novo módulo, com separação de responsabilidades. 19
Algoritmo 3 – Estrutura da rota de criação de solicitação. 26
Algoritmo 4 – Estrutura da entidade `Solicitation` e enumerações de domínio (pseudocódigo). 28
Algoritmo 5 – Fluxo de reversão (pseudocódigo). 34
Algoritmo 6 – Limpeza de tabelas transacionais entre testes (pseudocódigo). 43

SUMÁRIO

LISTA DE FIGURAS viii

LISTA DE QUADROS ix

LISTA DE TABELAS x

LISTA DE ABREVIATURAS E SIGLAS xi

LISTA DE SÍMBOLOS xi

LISTA DE ALGORITMOS xi

1 – INTRODUÇÃO 1

1.1 Objetivos 1

1.1.1 Objetivo geral 1

1.1.2 Objetivos específicos 2

1.2 Escopo e limitações 2

1.3 Procedimentos metodológicos 3

1.4 Contexto institucional 3

1.5 Organização do relatório 4

2 – EMBASAMENTO TEÓRICO 5

2.1 Arquitetura e Padrões de Aplicações Web 5

2.1.1 Cliente-Servidor 5

2.1.2 REST e HTTP 5

2.1.3 MVC e Separação de Responsabilidade (SoC) 5

2.1.4 Service Layer 6

2.1.5 Additive Change 6

2.1.6 OWASP e Segurança de Aplicações Web 6

2.2 Gestão de Projeto Ágil 7

2.2.1 OpenProject 7

2.2.2 Scrum 7

2.2.3 Extreme Programming (XP) 7

2.3 Tecnologias do Back-end 8

2.3.1 Flask e Blueprints 8

2.3.2 PostgreSQL 8

2.3.3 JSONB e Modelagem de Estado 8

2.3.4 SQLAlchemy e Unidade de Trabalho 8

2.3.4.1 Mapeamento Objeto-Relacional e Estratégias de Herança . 9
2.3.5 Validação de Dados com Marshmallow e Flasgger 9
2.4 Auditoria, Rastreabilidade e Versionamento 9
2.4.1 Auditoria e Rastreabilidade 9
2.4.2 Fluxo de Aprovação com Máquina de Estados 10
2.4.3 Snapshot e Reversão 10
2.4.4 Evento de Domínio e Domain-Driven Design 10
2.4.5 Timeline e Registro de Eventos 11
2.5 Front-end e Controle de Acesso 11
2.5.1 Vue.js 11
2.5.2 Nuxt 12
2.5.3 Gerenciamento de estado com Vuex 13
2.5.4 RBAC e Controle de Acesso 14
2.6 Verificação e Ambiente 15
2.6.1 Testes Automatizados com Pytest 15
2.6.2 Ambiente Reprodutível com Docker 16
2.7 Controle de Versão e Integração Contínua 16
2.8 Síntese do Capítulo 16

3 – ATIVIDADES REALIZADAS 17
3.1 Estudo da Arquitetura do Sistema Existente 17
3.1.1 Visão Geral do Sistema 17
3.1.2 Características do Código Existente e Decisão por Additive Change . 17
3.1.3 Análise das Regras de Negócio 20
3.1.4 Entendimento do Fluxo Antigo e Diagnóstico de Demanda 20
3.1.5 Levantamento de Requisitos 22
3.1.5.1 Requisitos Funcionais 22
3.1.5.2 Requisitos Não Funcionais 24
3.2 Desenvolvimento do Módulo de Solicitações 24
3.2.1 Endpoints 24
3.2.2 Integração com o Banco de Dados 26
3.2.3 Autorização e Controle de Acesso 30
3.2.4 O Módulo de Revisão e a Efetivação das Alterações 30
3.2.4.1 Separação de Responsabilidades e Controle de Acesso . . . 30
3.2.4.2 A Revisão como um Evento de Domínio (Domain Event) . 31
3.2.4.3 Execução das Regras de Negócio e Atomicidade 31
3.3 Implementação do Sistema de Snapshot e Auditoria 32
3.3.1 Entidade Snapshot e Modelo de Persistência 32
3.3.2 Reversão de Alterações 33
3.3.3 Linha do Tempo de Eventos 35

3.3.4 Rastreabilidade e Limitações . . . 35
3.4 Desenvolvimento das Interfaces Web . . . 36
3.4.1 Telas Criadas . . . 36
3.4.1.1 Listagem administrativa — pages/solicitations/index.vue 37
3.4.1.2 Listagem do solicitante — pages/solicitations/my_solicitation.vue 37
3.4.1.3 Detalhe e revisão — pages/solicitations/_id.vue . . . 37
3.4.2 Componentes Vue . . . 38
3.4.3 Integração frontend/backend . . . 39
3.4.4 RBAC e controle de acesso . . . 39
3.4.5 Síntese da seção . . . 40
3.5 Testes e Validação . . . 40
3.5.1 Construção da Infraestrutura de Testes . . . 40
3.5.2 Isolamento entre Testes . . . 42
3.5.2.1 Fixtures de Autenticação . . . 42
3.5.3 Testes de integração . . . 44
3.5.4 Processo de Integração Contínua . . . 44
3.5.4.1 Estrutura do Pipeline . . . 44
3.5.4.1.1 Estágio de lint . . . 44
3.5.4.1.2 Estágio de testes . . . 44
3.5.4.2 Gatilhos e Bloqueio de Merge . . . 45

4 – CONSIDERAÇÕES FINAIS . . . 46

Referências . . . 47

Apêndices 49

APÊNDICE A – Nome do apêndice . . . 50

APÊNDICE B – Nome do outro apêndice . . . 51

Anexos 52

ANEXO A – Nome do anexo . . . 53

ANEXO B – Nome do outro anexo . . . 54

1 INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio supervisionado realizado entre fevereiro e junho de 2026 no Laboratório Assert, ambiente de desenvolvimento do Polo de Inovação do Instituto Federal da Paraíba (IFPB). O estágio contou com bolsa concedida pela Agência de Inovação (NEO-IFPB), e as atividades concentraram-se na evolução do sistema Banco de Especialistas, plataforma destinada ao gerenciamento de especialistas e ao apoio à composição de equipes para projetos de Pesquisa, Desenvolvimento e Inovação (PD&I).

O Banco de Especialistas permite o cadastro, a classificação e a avaliação de candidatos com base em critérios como formação acadêmica, experiência profissional e participação em projetos. Nesse contexto, a plataforma reúne informações utilizadas na gestão dos especialistas e nos processos de seleção e composição de equipes vinculadas às iniciativas do Polo de Inovação.

Durante o estágio, identificou-se uma demanda relacionada à alteração de categoria dos especialistas. O procedimento então utilizado exigia que o interessado solicitasse a mudança por e-mail e que um administrador realizasse manualmente uma consulta SQL diretamente no banco de dados. A ausência de um fluxo formal de solicitações dificultava o acompanhamento das alterações e a recuperação do estado anterior dos registros, além de aumentar o risco de inconsistências decorrentes de intervenções diretas na base de dados.

Diante desse cenário, o trabalho concentrou-se no desenvolvimento de um módulo de solicitações para formalizar e gerenciar operações de alteração cadastral e de classificação dos especialistas. A solução contempla mecanismos para registrar solicitações, acompanhar seu andamento, permitir sua análise e decisão e manter informações necessárias à rastreabilidade das operações. Dessa forma, busca-se reduzir a dependência de intervenções manuais no banco de dados e estabelecer um processo mais controlado e verificável.

Do ponto de vista técnico, o desenvolvimento envolveu conceitos de engenharia de software, aplicações web, persistência de dados relacionais, testes automatizados e conteinerização. Destacam-se a integração entre componentes de front-end e back-end, o uso de mecanismos de registro e recuperação do estado dos usuários e a manutenção de um histórico das operações. Esses fundamentos são apresentados no capítulo dedicado ao embasamento teórico.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um módulo de solicitações de alteração de uso geral para o sistema Banco de Especialistas, contemplando os fluxos de criação, consulta, aprovação, rejeição e reversão, com mecanismos de rastreabilidade e registro das operações.

1.1.2 Objetivos específicos

- Levantar e analisar os requisitos, o contexto e as regras de negócio relacionados às solicitações de alteração cadastral e de classificação dos especialistas.
- Modelar os componentes de persistência necessários ao registro das solicitações e à recuperação do estado dos usuários.
- Implementar as funcionalidades de gerenciamento das solicitações no *back-end*, incluindo as regras de negócio, a integração com o PostgreSQL e o acesso aos dados por meio do SQLAlchemy.
- Desenvolver as interfaces web necessárias à criação, ao acompanhamento e à análise das solicitações.
- Implementar mecanismos de rastreabilidade das operações, incluindo o registro de eventos e a apresentação de seu histórico.
- Desenvolver e executar testes unitários e de integração para verificar as regras de negócio e o funcionamento integrado dos componentes da aplicação.
- Configurar o ambiente de integração contínua ao ambiente interno do IFPB, incluindo a execução automatizada de testes

1.2 Escopo e limitações

O trabalho compreende o desenvolvimento de um módulo de solicitações para o gerenciamento das operações de alteração cadastral e de classificação dos especialistas no sistema Banco de Especialistas. O escopo contempla o registro, o acompanhamento, a análise e a decisão das solicitações, incluindo os mecanismos de persistência e rastreabilidade necessários à manutenção do histórico das operações.

Nesse contexto, são consideradas duas modalidades de solicitação:

- **Complemento cadastral:** solicitação que submete o especialista a uma nova avaliação, alterando seu status para COMPLEMENTAÇÃO e desencadeando o recálculo de sua pontuação, conforme as regras de negócio vigentes.
- **Mudança de classificação:** solicitação de alteração da categoria do especialista entre estudante, colaborador externo e servidor. A operação está sujeita às regras de negócio do sistema, incluindo restrições à transição para a categoria atual e à aprovação quando já existe uma retificação em andamento.

O desenvolvimento não contempla a redefinição dos critérios de pontuação dos especialistas, a modificação do processo seletivo, a substituição do mecanismo de avaliação existente ou a implementação de notificações por e-mail. Também não faz parte do escopo o tratamento de múltiplas solicitações simultâneas para um mesmo especialista.

Quanto às limitações de execução, o trabalho foi desenvolvido por um único desenvolvedor, sob restrições de prazo e experiência. As decisões de arquitetura e implementação foram fundamentadas em pesquisa, documentação técnica e experimentação, sendo verificadas

por meio de testes automatizados e entregas incrementais. Nesse cenário, algumas escolhas priorizaram a simplicidade de implementação e a compatibilidade com a arquitetura legada, em detrimento de soluções mais abrangentes para cenários não contemplados pelos requisitos do projeto.

1.3 Procedimentos metodológicos

O trabalho caracteriza-se como um relato técnico de natureza aplicada e caráter descritivo, desenvolvido no contexto de um estágio supervisionado em desenvolvimento de software. Sua execução compreendeu o estudo da arquitetura e das regras de negócio do sistema Banco de Especialistas, o levantamento e a análise dos requisitos, a modelagem dos componentes necessários, a implementação das funcionalidades, a realização de testes e a construção do ambiente de integração contínua.

A verificação técnica envolveu testes unitários e de integração, destinados a avaliar o comportamento das funcionalidades implementadas e a interação entre os componentes da aplicação. Também foi verificado o funcionamento integrado da API em ambiente de desenvolvimento conteinerizado com Docker. As atividades foram conduzidas no contexto da arquitetura e das tecnologias já adotadas pelo projeto, considerando suas restrições.

As atividades do módulo foram organizadas no OpenProject (OpenProject, 2026), com issues, responsáveis e prazos, o que apoiou a priorização das tarefas e o acompanhamento do progresso ao longo do estágio.

Embora o Scrum seja amplamente difundido, durante a prática do estágio percebeu-se que o ritmo de trabalho e a natureza do projeto — desenvolvimento focado em um módulo de um sistema legado, conduzido por um desenvolvedor com apoio pontual de um technical lead, ambos com outras atribuições no projeto — aproximaram-se mais dos valores e práticas do Extreme Programming (XP). O foco não estava na execução de cerimônias rígidas de Sprints, mas nas práticas de engenharia do XP: pequenas entregas incrementais, testes automatizados rigorosos (Pytest) e integração contínua (CI). O OpenProject foi utilizado como ferramenta de gestão do backlog e das issues, permitindo priorização e monitoramento do fluxo de trabalho de forma adaptável.

1.4 Contexto institucional

O estágio foi concedido pela Agência de Inovação do Instituto Federal da Paraíba (NEO-IFPB) e realizado no âmbito do Polo de Inovação da instituição. O Polo atua no desenvolvimento de iniciativas de PD&I, aproximando a instituição de demandas tecnológicas do setor produtivo e da sociedade.

As atividades técnicas foram desenvolvidas no Laboratório Assert, ambiente vinculado ao Polo de Inovação no qual são realizadas atividades de desenvolvimento e manutenção de sistemas computacionais para projetos institucionais. Foi nesse laboratório que o estágio

proporcionou a participação nas etapas de análise, implementação, integração e verificação de funcionalidades do sistema Banco de Especialistas.

1.5 Organização do relatório

Além desta introdução, o relatório está organizado em três capítulos. O capítulo 2 apresenta os conceitos e as tecnologias que fundamentam o módulo. O capítulo 3 descreve as atividades do estágio, da análise dos requisitos à implementação e aos testes. O capítulo 4 reúne as considerações finais, com a síntese dos resultados, as limitações e as possibilidades de evolução do trabalho.

2 EMBASAMENTO TEÓRICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, a auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

2.1 Arquitetura e Padrões de Aplicações Web

2.1.1 Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em provedores de serviços e requisitantes (TANENBAUM; van Steen, 2007). Nesse contexto, a comunicação orientada pelo estilo REST e pelo protocolo HTTP define interações sem estado, nas quais cada requisição é independente e deve carregar as informações necessárias ao seu processamento (FIELDING; NOTTINGHAM; RESCHKE, 2022).

2.1.2 REST e HTTP

O estilo arquitetural REST (Representational State Transfer) define um conjunto de restrições que, quando aplicadas, promovem escalabilidade, simplicidade e independência entre cliente e servidor (FIELDING, 2000). A comunicação entre os componentes do sistema ocorre por meio de recursos identificados por URIs, manipulados por métodos HTTP (GET, POST, PUT, DELETE e etc.) e representados em formatos padronizados, como JSON ou XML. A aplicação dessas restrições permite que o cliente e o servidor evoluam de forma independente, desde que a interface entre eles permaneça consistente

2.1.3 MVC e Separação de Responsabilidade (SoC)

O padrão de arquitetura MVC (Model-View-Controller) organiza a aplicação em três componentes: o *Model* representa os dados e as regras de negócio, a *View* é responsável pela apresentação e interação com o usuário, e o *Controller* atua como intermediário, recebendo as requisições da interface, manipulando os dados e retornando respostas (GAMMA et al., 1994). A separação de responsabilidades (SoC - Separation of Concerns) permite que cada componente evolua independentemente, desde que a interface entre eles permaneça consistente.

Tabela 1 – Códigos HTTP, significados padrão e aplicações no módulo.

Código HTTP	**Significado padrão**	**Aplicação no módulo**
200	Sucesso	Consultas e operações concluídas com êxito.
201	Recurso criado	Criação de uma nova solicitação.
400	Requisição inválida	Erros de validação nos dados enviados.
401	Falha de autenticação	Credenciais inválidas.
403	Proibido	Tentativa de acesso a recurso sem permissão.
404	Não encontrado	Solicitação ou recurso inexistente.
409	Conflito de estado	Tentativa de transição inválida (por exemplo, reverter solicitação não aprovada).

Fonte: Elaboração própria, com base em RFC 9110 (FIELDING; NOTTINGHAM; RESCHKE, 2022).

2.1.4 Service Layer

O padrão Service Layer organiza a lógica de negócio em uma camada intermediária entre o *Controller* e o *Model*, promovendo a reutilização de regras de negócio e a centralização das operações que envolvem múltiplas entidades (FOWLER, 2006). Essa abordagem facilita a manutenção do sistema, pois as alterações nas regras de negócio podem ser implementadas em um único local, sem impactar diretamente os controladores ou modelos individuais.

2.1.5 Additive Change

O padrão Additive Change descreve a evolução de um sistema por meio da adição de novas funcionalidades, em vez de modificar diretamente o código existente (FOWLER, 2004). Essa abordagem minimiza o risco de introduzir erros em funcionalidades já implementadas e permite que o sistema cresça de forma incremental, preservando a estabilidade e a confiabilidade das operações existentes. No contexto do módulo de solicitações, o Additive Change foi aplicado ao desenvolver novas rotinas para criação, análise e reversão de solicitações, sem alterar as funcionalidades já presentes no Banco de Especialistas.

2.1.6 OWASP e Segurança de Aplicações Web

O Open Worldwide Application Security Project (OWASP) mantém o OWASP Top 10, uma lista de referência global que elenca os dez riscos de segurança mais críticos para aplicações web (GLAS et al., 2025). A edição mais recente (2025) destaca que o Controle de Acesso Quebrado (Broken Access Control) permanece no topo do ranking como o risco mais sério, seguido por Configuração Insegura (Security Misconfiguration). Para um sistema que lida com dados de usuários e decisões administrativas, como o Banco de Especialistas, alinhar o desenvolvimento a esse referencial não é apenas uma boa prática, mas uma necessidade para mitigar vulnerabilidades exploráveis.

2.2 Gestão de Projeto Ágil

A organização das atividades de desenvolvimento também exigiu atenção à gestão do projeto, as prioridades e ao acompanhamento do progresso. Em contextos de software, a gestão ágil busca reduzir a incerteza por meio de entregas incrementais, feedback frequente e adaptação contínua às mudanças, em vez de dependência excessiva de planos rígidos e de documentação extensa (COHN, 2009). Nesse sentido, a gestão do módulo foi apoiada por ferramentas e práticas que permitiram organizar o backlog, acompanhar pendências e controlar o avanço das tarefas de forma transparente.

2.2.1 OpenProject

O OpenProject é uma plataforma de gestão de projetos e de colaboração que oferece recursos de backlog, planejamento, acompanhamento de tarefas, cronogramas e relatórios de progresso (OpenProject, 2026). No estágio, ele foi utilizado para registrar issues, atribuir responsabilidades, definir prioridades e monitorar o avanço das atividades relacionadas ao desenvolvimento do módulo. Essa ferramenta contribuiu para manter o trabalho organizado, reduzindo a perda de contexto e permitindo uma visão clara do status das pendências em cada etapa do projeto.

2.2.2 Scrum

O Scrum é uma das abordagens ágeis mais conhecidas e organiza o trabalho em ciclos curtos, chamados *sprints*, com papéis definidos, backlog priorizado e cerimônias de planejamento, revisão e retrospectiva (COHN, 2009). A estrutura do Scrum é especialmente útil em equipes maiores e em projetos em que a mudança de requisitos é frequente, pois favorece a entrega incremental de valor e a revisão contínua do produto. No caso do estágio, a metodologia não foi seguida em sua forma estrita, mas os princípios de priorização, transparência e acompanhamento incremental foram relevantes para organizar o desenvolvimento do módulo.

2.2.3 Extreme Programming (XP)

O Extreme Programming (XP) enfatiza práticas de engenharia que aumentam a qualidade do software e a capacidade de responder rapidamente às mudanças de requisitos (BECK; ANDRES, 2004). Entre suas práticas mais conhecidas estão pequenas entregas, feedback frequente, desenvolvimento orientado a testes, integração contínua, refatoração e foco em simplicidade. Como o módulo foi implementado em um sistema legado, com regras de negócio bem definidas e necessidade de validação constante, o XP se mostrou mais alinhado ao ritmo de trabalho do que a execução rígida de cerimônias do Scrum. As entregas menores, os testes automatizados e a revisão contínua reduziram o risco de regressão e facilitaram a adaptação à estrutura já existente do Banco de Especialistas.

A combinação de priorização em backlog, uso do OpenProject para o acompanhamento das atividades e adoção de práticas do XP permitiu manter um fluxo de trabalho disciplinado, mesmo em um contexto de desenvolvimento com escopo específico e um único responsável técnico. Essa abordagem equilibrou organização gerencial e rigor técnico, o que foi fundamental para a construção do módulo de solicitações.

2.3 Tecnologias do Back-end

2.3.1 Flask e Blueprints

O Flask foi utilizado na API do módulo por permitir incorporar suas rotas à aplicação existente sem impor uma reorganização do sistema. Os *blueprints* agruparam as operações de criação, análise, acompanhamento e reversão das solicitações em um componente isolado, preservando as demais funcionalidades e facilitando a manutenção do módulo (PALLETS, 2024).

2.3.2 PostgreSQL

O módulo utiliza PostgreSQL para armazenar solicitações, alterações cadastrais e registros de auditoria. A persistência dessas informações precisa manter o cadastro e o histórico coerentes, sobretudo quando uma decisão altera mais de uma entidade.

O PostgreSQL foi adequado por oferecer transações para as operações compostas do fluxo. Na aprovação, a mudança de categoria, a atualização da pontuação, a gravação do *snapshot* do estado anterior e a atualização do status da solicitação devem ser confirmadas em conjunto; se uma etapa falhar, nenhuma alteração deve permanecer aplicada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024).

2.3.3 JSONB e Modelagem de Estado

O PostgreSQL oferece o tipo de dado JSONB, que armazena documentos JSON de forma binária, permitindo consultas e indexação eficientes (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024). O módulo utiliza esse recurso para registrar o estado do especialista no momento da solicitação, preservando a categoria, a pontuação e outros atributos relevantes. Essa abordagem facilita a reversão de alterações e a auditoria das mudanças, mantendo o histórico completo das operações realizadas.

2.3.4 SQLAlchemy e Unidade de Trabalho

O SQLAlchemy foi adotado como ORM para coordenar as alterações de entidades relacionadas em uma única unidade de trabalho (BAYER, 2012). A sessão acumula as mudanças e, no *commit*, efetiva-as na transação; em caso de falha, o *rollback* descarta o conjunto. Assim,

a aprovação não deixa a categoria atualizada sem o *snapshot* correspondente ou sem o novo status da solicitação.

2.3.4.1 Mapeamento Objeto-Relacional e Estratégias de Herança

O SQLAlchemy oferece diferentes estratégias de herança para mapear hierarquias de classes. A estratégia Joined Table Inheritance (herança de tabela unida) cria tabelas separadas para a classe base e suas subclasses, sendo útil quando há muitos atributos exclusivos. Já a estratégia Single Table Inheritance (herança de tabela única) armazena os diferentes tipos de entidades em uma única tabela, utilizando uma coluna discriminadora. No módulo de solicitações, adotou-se o Single Table Inheritance para os tipos de solicitação (mudança de categoria, recurso e pontuação), pois eles compartilham a maior parte dos atributos, o que evita junções complexas e otimiza as consultas de listagem.

2.3.5 Validação de Dados com Marshmallow e Flasgger

Na fronteira da API, os *schemas* do Marshmallow validam os campos e formatos recebidos antes que os dados cheguem às regras de negócio (MARSHMALLOW DEVELOPERS, 2024). Ao declarar os formatos esperados para as solicitações e suas respostas, também estabelecem o contrato consumido pelas interfaces de criação, acompanhamento e análise do módulo.

2.4 Auditoria, Rastreabilidade e Versionamento

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

2.4.1 Auditoria e Rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (FOWLER, 2004) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como

uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

2.4.2 Fluxo de Aprovação com Máquina de Estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada e rejeitada) e de transições permitidas entre eles (SOMMERVILLE, 2011). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

2.4.3 Snapshot e Reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (GAMMA et al., 1994), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (FOWLER, 2005; KLEPPMANN, 2017). Embora o Event Sourcing seja uma alternativa robusta, o módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

2.4.4 Evento de Domínio e Domain-Driven Design

O Domain-Driven Design (DDD) é uma abordagem de design de software que coloca o domínio do negócio no centro do desenvolvimento, utilizando uma linguagem ubíqua para alinhar especialistas de negócio e desenvolvedores (EVANS, 2003). Dentro do DDD, o conceito de Evento de Domínio (Domain Event) representa algo significativo que aconteceu no domínio do problema. Um evento de domínio é, por definição, um registro imutável de um fato passado

(por exemplo, "Solicitação Aprovada"ou "Categoria Alterada"), que captura a intenção e o contexto da operação (EVANS, 2003).

No contexto do Banco de Especialistas, o módulo de revisão foi modelado sob essa ótica. A entidade SolicitationReview não atua apenas como uma tabela de auditoria, mas como o registro do Evento de Domínio que materializa a decisão tomada pelo avaliador. Ao registrar o evento (a revisão) de forma imutável e com autoria definida, o sistema garante que as transições de estado do especialista (como a mudança de categoria) sejam disparadas a partir de um fato consumado e rastreável, promovendo o desacoplamento entre a intenção (solicitação) e a sua efetivação (evento de aprovação).

2.4.5 Timeline e Registro de Eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

2.5 Front-end e Controle de Acesso

As interfaces do módulo de solicitações foram construídas sobre a arquitetura já adotada pelo Banco de Especialistas, baseada em Vue.js, Nuxt e Vuetify. Este capítulo apresenta os conceitos que sustentam essa camada: o modelo de reatividade e composição do Vue.js, a organização de aplicações universais do Nuxt, o gerenciamento de estado com Vuex e o controle de acesso baseado em papéis (RBAC). A compreensão desses fundamentos é necessária porque as decisões de implementação descritas na Seção **??** — como a divisão entre listagem administrativa e listagem do solicitante, o uso de módulos Vuex por domínio e a exibição condicional do painel de revisão — não são arbitrárias: decorrem das propriedades dessas tecnologias e do modelo de segurança adotado.

2.5.1 Vue.js

O Vue.js é um *framework* progressivo para construção de interfaces de usuário. O termo *progressivo* designa uma característica central da biblioteca: ela pode ser adotada de forma incremental, desde o uso pontual em uma página existente até a estruturação completa de uma *single-page application* (SPA) (VUE.JS, 2024). Essa propriedade é relevante no contexto do Banco de Especialistas, cujo front-end legado já estava consolidado em Vue.js; o módulo novo pôde ser incorporado sem exigir uma reescrita da camada de apresentação, em coerência com o princípio de *Additive Change* discutido na Seção 2.1.5.

O núcleo do Vue.js é o **sistema de reatividade**. Ao declarar dados no estado de um componente (`data`), o framework estabelece uma relação de dependência entre esses dados e os

pontos da interface que os consomem. Quando o valor muda, o Vue identifica os componentes afetados e atualiza apenas as partes da árvore de renderização que dependem daquele dado, por meio de um *virtual DOM*. Para o desenvolvedor, isso significa que a interface não precisa ser manipulada manualmente: a declaração do estado e a ligação (*binding*) com a visualização bastam para que a atualização ocorra. No módulo de solicitações, essa propriedade sustenta comportamentos como a reação ao retorno de uma decisão de revisão — o estado local do componente é atualizado e a interface reflete o novo status sem que seja necessário recarregar a página (Seção **??**).

O Vue.js organiza a interface em **componentes**, unidades reutilizáveis que encapsulam estrutura, comportamento e estilo. No formato de *Single-File Components* (SFC), cada componente é um arquivo `.vue` que reúne três blocos: `<template>`, com a marcação declarativa da interface; `<script>`, com a lógica de estado e comportamento; e `<style>`, com a apresentação visual. Essa coesão por componente é o que permite ao módulo isolar o formulário de criação em um componente próprio (`SolicitationModal`), reutilizável em telas distintas, e reaproveitar componentes legados como o `PreviewDocumentDialog` sem alterá-los (Seção **??**).

A comunicação entre dados e interface é mediada por **diretivas**, atributos especiais aplicados no *template* que expressam comportamentos reativos. O `v-model` estabelece ligação bidirecional entre um campo de formulário e uma propriedade do componente; `v-if` e `v-show` condicionam a renderização à avaliação de uma expressão; `v-for` itera sobre coleções. No módulo, essas diretivas são empregadas de forma sistemática: `v-model` sustenta todos os campos do formulário de criação; `v-if` controla a exibição condicional do campo "Nova Categoria" quando o tipo de solicitação é mudança de categoria; e `v-for` percorre a lista de solicitações na renderização dos *cards*. A combinação dessas diretivas com o sistema de reatividade é o que torna o formulário adaptável ao tipo de pedido sem duplicação de código.

Por fim, cabe distinguir as duas *APIs* de composição do Vue.js. A **Options API**, adotada no projeto, organiza o componente em propriedades nomeadas (`data`, `methods`, `computed`, `created`, `watch`), o que torna explícita a separação entre estado, comportamento e ciclo de vida. A **Composition API**, introduzida em versões posteriores, permite agrupar a lógica por funcionalidade. A escolha da Options API no módulo não é técnica, mas de compatibilidade: o restante da aplicação a utiliza, e manter o mesmo estilo preserva a consistência e reduz a curva de aprendizado para quem for dar manutenção ao código.

2.5.2 Nuxt

O Nuxt é um *meta-framework* construído sobre o Vue.js que estrutura a aplicação e adiciona funcionalidades ausentes no *framework* base. A principal delas é o **modo universal**: a aplicação pode ser renderizada no servidor (*Server-Side Rendering*, SSR) e depois assumir o controle no cliente (*Client-Side Rendering*, CSR), resultando em uma experiência híbrida (NUXT, 2024). O SSR entrega o HTML já populado na primeira requisição, o que melhora o tempo de primeira renderização percebida e a indexação por mecanismos de busca; a partir daí,

a navegação subsequente ocorre como em uma SPA, sem recarregamento completo da página. Para o Banco de Especialistas, esse modelo é vantajoso porque a plataforma tem finalidade institucional e precisa ser acessível e indexável, mas também oferece interações intensas — como os formulários de solicitação e o painel de revisão — que se beneficiam da fluidez de uma SPA.

O Nuxt impõe uma **estrutura de diretórios** que organiza a aplicação por responsabilidade. Os diretórios centrais são: `pages/`, que define as rotas por convenção de nome de arquivo; `components/`, que reúne componentes reutilizáveis; `layouts/`, que define a estrutura visual compartilhada; `store/`, que abriga os módulos de gerenciamento de estado (discutidos na Seção 2.5.3); e `plugins/`, que registra extensões do *framework* e do cliente HTTP. Essa organização é adotada integralmente no módulo: as telas ficam em `pages/solicitations/`, o modal em `components/users/SolicitationModal.vue` e os módulos de estado em `store/specialists` e `store/appeal`. A estrutura convencional reduz decisões de organização e garante que qualquer novo desenvolvedor encontre os artefatos nos lugares esperados — propriedade relevante em um contexto de manutenção por equipe.

O **roteamento por arquivos** é outra característica central. Em vez de declarar rotas em um arquivo de configuração, o Nuxt as deriva da estrutura de `pages/`: o arquivo `pages/solicitations/index.vue` corresponde à rota `/solicitations`, e `pages/solicitations/_id.v` corresponde a `/solicitations/:id`, com o parâmetro dinâmico identificado pelo sublinhado. As três telas do módulo são mapeadas por essa convenção (Seção **??**), e a navegação entre elas é feita por `$router.push` a partir do identificador do registro. A convenção também explicita a hierarquia de recursos: o detalhe de uma solicitação é uma subrota da listagem, o que reflete a relação entre os recursos na API (`/solicitations` e `/solicitations/id`).

O acesso à API é mediado pelo módulo `@nuxtjs/axios`, que injeta um cliente HTTP no contexto da aplicação e o disponibiliza em componentes e *stores* por meio de `this.$axios`. No módulo, esse cliente é usado tanto diretamente nas telas (por exemplo, em `getDetailSolicitation`, que consulta `/solicitations/id`) quanto nos módulos Vuex (que encapsulam chamadas para `/appeals` e `/solicitations`). A centralização do cliente HTTP permite configurar cabeçalhos, tokens de autenticação e interceptadores de erro em um único lugar, o que reduz a duplicação e padroniza o tratamento de falhas na camada de interface.

2.5.3 Gerenciamento de estado com Vuex

A medida que a interface cresce, o estado compartilhado entre componentes deixa de caber no escopo local de cada um. O Vuex resolve esse problema ao centralizar o estado da aplicação em um *store* único, com regras explícitas de leitura e escrita (**??**). A arquitetura do Vuex organiza o fluxo de dados em quatro conceitos: o **state**, que armazena os dados; os **getters**, que expõem leituras derivadas; as **mutations**, que alteram o estado de forma síncrona e rastreável; e as **actions**, que orquestram operações assíncronas — tipicamente chamadas à API — e, ao final, disparam *mutations* para atualizar o estado. Essa disciplina

evita que múltiplos componentes alterem o mesmo dado de formas conflitantes e torna o fluxo de mudança de estado previsível.

O Vuex permite dividir o *store* em **módulos com namespace**, cada um responsável por um domínio da aplicação. O módulo de solicitações faz uso extensivo desse recurso: `store/specialists` expõe ações como `getSolicitations`, consumida pelas telas de listagem; e `store/appeal` isola as operações de recurso (`getAppealsListByUserId`, `createAppeal`, `completeAppeal`). A separação por domínio mantém as telas focadas em apresentação e delega o acesso a dados e a transformação de estado ao *store*, o que reduz o acoplamento entre interface e API. A nomenclatura segue a convenção do Nuxt, em que o nome do arquivo define o *namespace* do módulo, e as ações são despachadas com `mapActions` ou com o prefixo do domínio.

No contexto do módulo, a adoção do Vuex não é apenas organizacional: ela é o que permite que a listagem e a criação compartilhem o mesmo estado de solicitações. Quando o modal de criação conclui o envio com sucesso, a listagem do solicitante pode refletir o novo registro sem que os dois componentes precisem se conhecer diretamente — o *store* atua como intermediário. O mesmo princípio se aplica à atualização de status após uma decisão de revisão: a tela de detalhe atualiza o estado local e, se necessário, o *store*, garantindo que a próxima navegação encontre os dados atualizados.

2.5.4 RBAC e Controle de Acesso

O controle de acesso baseado em papéis (*Role-Based Access Control*, RBAC) é um modelo de segurança que restringe o acesso a recursos com base nas funções atribuídas aos usuários, e não em identidades individuais (SANDHU et al., 1996). A ideia central é a separação entre **usuários**, **papéis** e **permissões**: usuários são associados a papéis; papéis agregam permissões; e permissões autorizam operações sobre recursos. Essa indireção simplifica a administração de segurança, porque a concessão ou revogação de um privilégio é feita sobre o papel, e não sobre cada usuário individualmente. Em um sistema que lida com dados de pessoas e decisões administrativas, como o Banco de Especialistas, esse modelo é particularmente adequado: os perfis de acesso são poucos e bem definidos, e a política de autorização precisa ser auditável.

A escolha do RBAC também se justifica pelo alinhamento com o OWASP Top 10. A edição mais recente do ranking mantém o *Broken Access Control* na primeira posição dos riscos mais críticos para aplicações web (**??**), o que reforça a necessidade de um modelo de autorização explícito, verificado no servidor e auditável. O módulo adota essa orientação em duas camadas complementares. A camada **autoritativa** está no servidor (Seção **??**): cada rota declara o perfil exigido por meio de um decorador (`@auth.login_required`), e o serviço verifica se o solicitante tem acesso ao recurso específico. A camada de **interface** apenas reflete, para o usuário, o que o servidor já garante — ocultando operações para as quais o usuário não tem permissão, mas sem substituir a verificação no *backend*. Essa divisão é essencial:

a ocultação de um botão é conveniência de usabilidade, não mecanismo de segurança; um usuário sem perfil que tente chamar a rota diretamente recebe 403 do servidor, preservando a integridade do fluxo.

No módulo, os papéis não são mutuamente exclusivos: um mesmo usuário pode ser solicitante e, ao mesmo tempo, possuir a permissão de revisão, dependendo dos perfis e das *features* associados à sua conta. A autorização é expressa, portanto, não apenas por papéis, mas pela combinação entre **perfil** (`ProfileEnum.ADMIN`) e **permissão** (`FeatureEnum.SPECIALIST_REVIEW`). O Quadro 2 sintetiza as operações disponíveis no módulo e as condições que cada uma exige. As definições refletem o que está implementado no servidor (Seção **??**); a interface apenas respeita essas restrições.

Tabela 2 – Papéis e permissões no módulo de solicitações.

Operação	**Solicitante**	**Avaliador**	**Administrador**	**Requisito**
Criar solicitação	Sim	Sim	Sim	RF01–RF03
Listar próprias solicitações	Sim	Sim	Sim	RF04
Listar todas as solicitações	Não	Sim	Sim	RF04
Consultar detalhe (própria)	Sim	Sim	Sim	RF04
Consultar detalhe (qualquer)	Não	Sim	Sim	RF04
Aprovar ou rejeitar	Não	Sim	Sim	RF05–RF07
Reverter alteração aprovada	Não	Não	Sim	RF09

Fonte: Elaboração própria (2026).

A definição dos papéis operacionais é a seguinte. O **solicitante** é qualquer usuário autenticado que cria solicitações e acompanha as suas próprias; não tem acesso à listagem geral nem à decisão de revisão. O **avaliador** é o usuário que possui a *feature* `SPECIALIST_REVIEW`, o que lhe permite listar todas as solicitações, consultar o detalhe de qualquer uma e proferir decisão de aprovação ou rejeição. O **administrador** acumula o perfil `ProfileEnum.ADMIN` e a *feature* de revisão, e é o único habilitado a executar a reversão de uma alteração aprovada — operação de efeito sobre o cadastro do especialista que exige, pela sua gravidade, o nível mais alto de autorização. A tabela também explicita o que cada papel *não* pode fazer: a coluna de requisito permite rastrear, para cada operação, o requisito funcional correspondente no Quadro **??** da Seção **??**, o que fecha a cadeia de rastreabilidade entre requisito, papel, operação e verificação.

2.6 Verificação e Ambiente

2.6.1 Testes Automatizados com Pytest

O Pytest foi empregado para verificar as regras de negócio do módulo, em especial as transições permitidas da máquina de estados: criação, aprovação, rejeição e reversão (HOLGER KREKEL AND PYTEST DEVELOPERS, 2024). Os testes também verificaram os efeitos dessas transições sobre a categoria, a pontuação e o histórico da solicitação.

2.6.2 Ambiente Reprodutível com Docker

O Docker foi utilizado para reproduzir o ambiente necessário aos testes de integração, incluindo a API e o PostgreSQL, de modo que a validação dos fluxos ocorresse sob uma configuração consistente entre execuções (DOCKER, 2024).

2.7 Controle de Versão e Integração Contínua

A gestão de configuração de software compreende a identificação, o controle e o registro das alterações realizadas nos artefatos de um projeto ao longo do tempo, de modo que seja possível reconstituir versões anteriores e identificar a origem de cada mudança (SOMMERVILLE, 2011). O controle de versão é o mecanismo que concretiza esse registro sobre o código-fonte: cada alteração é capturada em um commit, com autor, data e descrição, e o histórico resultante permite recuperar estados anteriores do sistema.

O módulo utilizou Git (Git, 2026) para esse fim, com ramificações (branches) por funcionalidade, o que isolou o desenvolvimento do módulo das demais partes do Banco de Especialistas até que as alterações fossem verificadas. Os repositórios foram hospedados no GitLab (GitLab, 2026), adotado como padrão pelo Laboratório Assert para os projetos do Polo de Inovação. A integração contínua automatiza a execução de verificações, como a suíte de testes Pytest, a cada commit enviado ao repositório; neste projeto, essa execução é realizada por uma pipeline, impedindo que alterações que quebrem as regras de negócio sejam incorporadas à base principal.

Cabe observar que o controle de versão atua sobre o código-fonte, e não sobre os dados da aplicação. A rastreabilidade das alterações cadastrais dos especialistas, tratada na seção 2.4, é um problema distinto, resolvido no domínio da aplicação por meio de solicitações, snapshots e registro de eventos.

2.8 Síntese do Capítulo

As tecnologias e os conceitos apresentados compõem a base sobre a qual o módulo de solicitações foi desenvolvido. A arquitetura cliente-servidor e o estilo REST definem a forma de comunicação entre as interfaces e a API; o Flask estrutura a aplicação, o SQLAlchemy e o PostgreSQL garantem a persistência transacional, e o Marshmallow assegura a validação e o contrato dos dados. Os conceitos de auditoria, máquina de estados, *snapshot* e registro de eventos sustentam os requisitos de rastreabilidade e reversão que motivaram o estágio. Por fim, o Pytest garante a verificação das regras de negócio, o Git organiza a colaboração e o Docker fornece o ambiente reprodutível de execução. O próximo capítulo descreve como esses elementos foram aplicados nas atividades realizadas.

3 ATIVIDADES REALIZADAS

Este capítulo apresenta as principais atividades realizadas no decorrer do estudo e do desenvolvimento da funcionalidade de alteração de categoria de especialistas. A partir da análise do sistema existente e do levantamento das regras de negócio, foi possível compreender o contexto institucional, os fluxos de operação e os requisitos necessários para a implementação de uma solução integrada, auditável e compatível com a arquitetura atual.

3.1 Estudo da Arquitetura do Sistema Existente

3.1.1 Visão Geral do Sistema

O primeiro passo para o desenvolvimento da funcionalidade consistiu na compreensão do sistema Banco de Especialistas, de sua finalidade institucional e de sua organização técnica. A plataforma permite o cadastramento e a classificação de profissionais interessados em participar de projetos de pesquisa, desenvolvimento e inovação vinculados ao IFPB.

Nesse contexto, foram estudadas as funcionalidades relacionadas ao cadastro dos especialistas, à organização por categorias e aos critérios de classificação utilizados pelo sistema. Também foram examinados os componentes da aplicação e suas interações, com o objetivo de identificar os pontos de integração necessários ao desenvolvimento do novo módulo.

3.1.2 Características do Código Existente e Decisão por Additive Change

A adoção do princípio de Additive Change foi uma decisão arquitetural relevante para garantir a compatibilidade com a base já existente. Esse princípio orienta a evolução do sistema por meio da adição de novas rotas, campos, serviços e módulos, sem a necessidade de reescrever estruturas centrais ou alterar de forma profunda o comportamento de componentes já consolidados.

Essa estratégia se mostrou adequada ao contexto do projeto, pois a aplicação legada foi estruturada em um padrão MVC com controladores responsáveis por múltiplas tarefas, incluindo validação, persistência, tratamento de exceções, registro de eventos e comunicação com camadas externas. Esse desenho favoreceu a rapidez inicial de implementação, mas também resultou em baixa coesão, acoplamento funcional e maior dificuldade de manutenção e evolução.

A partir dessa análise, a nova funcionalidade foi concebida como uma extensão incremental do sistema, preservando os comportamentos já consolidados e adicionando um fluxo específico para a solicitação, aprovação, auditoria e registro histórico das mudanças de categoria. Essa abordagem reduziu o risco de regressão e permitiu a inserção de novos mecanismos sem comprometimento da infraestrutura preexistente.

A seguir, é apresentado o padrão observado em controladores existentes, evidenciando as características que motivaram a escolha por uma evolução incremental.

Algoritmo 1: Padrão de implementação observado em controladores existentes.

```
@bp.route('', methods=['POST'])
@swag.validate('UserRegistration')
def register():
    if not validate_cpf(request.json['cpf']):
        raise ValidationError(details={'cpf': 'invalid'})

    user = User(**request.json)
    user.hash_password()

    try:
        db.session.add(user)
        db.session.commit()
    except IntegrityError as e:
        msg = e.args[0]
        if 'users_tipo_especialista_fkey' in msg:
            raise InvalidIdFieldError('specialist_type_id')
        elif 'users_cpf_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('cpf')
        elif 'users_email_key' in msg:
            raise UniqueIdentifierAlreadyExistsError('email')
        else:
            raise e

    create_timeline_log(
        activity_type_id=ActivityTypeEnum.CADASTRADO,
        user_id=user.id)

    msg = 'Informe seus dados pessoais...'
    create_notification(title='Boas vindas!', message=msg, user=user)

    return UserSchema().dump(user), HTTPStatus.CREATED
```

O padrão exemplificado no Algoritmo 1 demonstra a forma como os controladores legados eram organizados. Em uma mesma função, eram concentradas validação de entrada, construção do objeto de domínio, hash de senha, persistência no banco de dados, tratamento de erros de integridade, registro de timeline, envio de notificação e serialização da resposta. A validação dos dados de entrada era realizada pelo Flasgger, por meio do decorador `@swag.validate`, que verificava os dados conforme o schema associado à operação. Assim, a documentação Swagger também era utilizada como base para a validação das requisições.

Esse desenho gera alguns acoplamentos relevantes. O primeiro deles está no tratamento de erro, em que a lógica depende de strings de constraint do banco, como `users_tipo_especialista_fkey` e `users_cpf_key`. Esse tipo de condição cria uma forte dependência com o esquema físico do banco de dados, de forma que qualquer renomeação ou

alteração da constraint pode quebrar o fluxo de tratamento de exceções. O segundo acoplamento está na execução de operações colaterais após o commit, como a criação de registros de timeline e notificações. Nesse cenário, a criação do usuário e os eventos associados não são tratados como uma unidade transacional, de modo que falhas nas etapas posteriores podem deixar um registro principal persistido sem os dados complementares esperados.

A comparação com o novo módulo evidencia a importância da abordagem incremental. O desenvolvimento do módulo de solicitações foi organizado para separar responsabilidades e encapsular regras de negócio em serviços e camadas mais específicas, preservando o restante da aplicação e reduzindo o risco de efeitos colaterais.

Algoritmo 2: Estrutura do novo módulo, com separação de responsabilidades.

```
@bp_solicitacoes.route('', methods=['POST'])
@schema_required(SolicitacaoCreateSchema)
def criar_solicitacao(payload):
    solicitacao = solicitacao_service.criar(
        solicitante_id=current_user.id,
        tipo=payload['tipo'],
        dados=payload['dados'],
    )
    return SolicitacaoSchema().dump(solicitacao), HTTPStatus.CREATED


def criar(solicitante_id, tipo, dados):
    campos = SCHEMA_POR_TIPO[tipo].load(dados)
    solicitacao = Solicitacao(...)

    with db.session.begin_nested():
        db.session.add(solicitacao)
        registrar_evento(solicitacao, Evento.CRIADA)

    return solicitacao
```

O Algoritmo 2 ilustra o padrão adotado no novo módulo. A validação ocorre na fronteira, por meio do decorador `@schema_required(SolicitacaoCreateSchema)`, de modo que o serviço recebe um payload já validado. O controlador limita-se a delegar a execução ao serviço e serializar a resposta, sem concentrar hash, tratamento de erro do banco ou envio de notificações. A seleção do schema por tipo, por meio de `SCHEMA_POR_TIPO[tipo].load(dados)`, representa o mecanismo das restrições dinâmicas descritas na análise de regras de negócio e reforça a separação entre validação e persistência. Por fim, a persistência e o registro do evento são agrupados em uma mesma unidade de trabalho, eliminando o intervalo entre o commit e os efeitos colaterais observado no Algoritmo 1.

A escolha por Additive Change permitiu incorporar o novo fluxo sem reescrever os controladores existentes, preservando o comportamento consolidado da aplicação.

3.1.3 Análise das Regras de Negócio

O sistema possui um modelo de regras de negócio relacionadas à classificação de especialistas e à alteração de seus dados cadastrais. Condições essas que deveriam ser respeitadas pela nova funcionalidade, bem como os efeitos das operações sobre os registros existentes.

Um dos principais desafios identificados decorre da organização do modelo de dados da aplicação: a categoria do especialista é definida no modelo *User*, e não diretamente no modelo *Specialist*. Embora os especialistas compartilhem uma estrutura de usuário comum, suas categorias determinam especializações distintas, com atributos específicos para cada tipo. Essa organização exige que as operações de transição considerem as relações entre os modelos envolvidos, de modo a preservar a integridade dos dados durante a alteração cadastral.

Outro aspecto relevante consiste em assegurar a rastreabilidade das alterações e a possibilidade de preservar o estado anterior dos registros durante o fluxo de solicitação. Para isso, é necessário distinguir o registro de uma solicitação da efetivação da mudança, condicionando esta última ao resultado do processo de análise. Adicionalmente, as transições devem considerar os critérios de classificação e pontuação aplicáveis a cada categoria, a fim de evitar inconsistências decorrentes da alteração dos dados cadastrais.

3.1.4 Entendimento do Fluxo Antigo e Diagnóstico de Demanda

Antes do desenvolvimento do módulo, as solicitações de mudanças de categoria e de atualização cadastral eram realizadas exclusivamente por e-mail. Após o recebimento do pedido, a alteração dependia da intervenção de um técnico, que executava manualmente as operações diretamente na base de dados por meio de scripts SQL reutilizáveis.

Esse procedimento estabelecia uma forte dependência entre a solicitação do usuário e a disponibilidade de um responsável com acesso privilegiado ao banco de dados. Além disso, o acompanhamento dos pedidos, a validação das justificativas e o histórico das alterações ficavam dispersos nas caixas de entrada de e-mail, fora de um fluxo centralizado e auditável.

Esse procedimento estabelecia uma dependência entre a solicitação do usuário e a disponibilidade de um responsável com acesso ao banco. Além disso, o acompanhamento dos pedidos, a validação das justificativas e o histórico das alterações ficavam dispersos na caixa de entrada do suporte, fora de um fluxo centralizado e auditável.

Para dimensionar a recorrência desse processo, realizou-se o levantamento dos chamados recebidos pela caixa de suporte no período de 30 de maio de 2025 a 4 de agosto de 2026. A catalogação foi feita manualmente pela equipe, a partir do assunto e do conteúdo das mensagens, resultando em 141 chamados relacionados à gestão de cadastro e de acesso dos especialistas. A Tabela 3 apresenta a distribuição conforme o tipo de vínculo do usuário.

A concentração de 56,0% em colaboradores externos é relevante: trata-se do público com menor familiaridade com os procedimentos internos do IFPB e, portanto, mais dependente de um canal formal e autoexplicativo para solicitar ajustes.

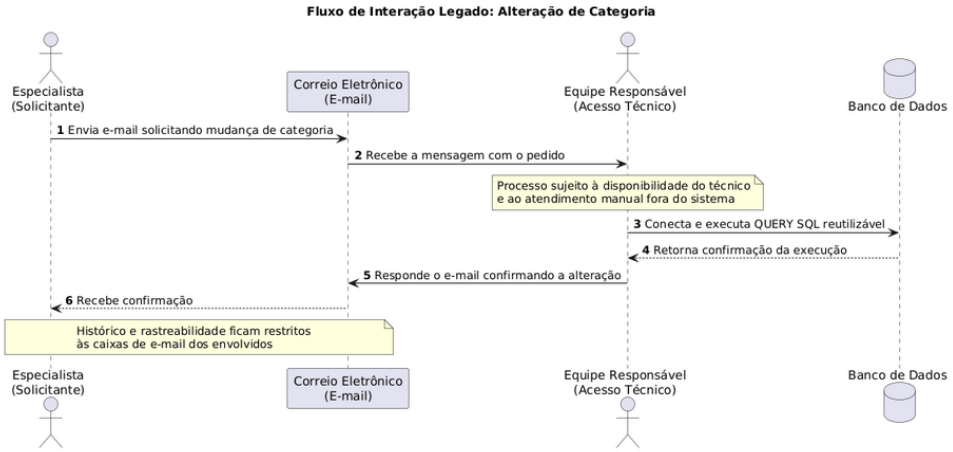


Figura 1 – Fluxo de solicitações manuais de ajuste cadastral e de categoria antes do desenvolvimento do módulo.

Fonte: Elaboração própria (2026).

Tabela 3 – Volume de solicitações de suporte por tipo de inscrição (maio/2025 a agosto/2026).

Tipo de Inscrição	**Quantidade de Chamados**	**Porcentagem (%)**
Colaborador(a) externo	79	56,0%
Estudante	47	33,3%
Servidor(a)	15	10,6%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

A Tabela 4 detalha os motivos apresentados nesses atendimentos.

Tabela 4 – Distribuição das solicitações de suporte por categoria de problema.

Categoria do Problema	**Quantidade**	**Porcentagem (%)**
Outro (especificar)	59	41,8%
Completar cadastro (dados pessoais)	34	24,1%
Cadastrar pontuação	34	24,1%
Cadastrar no sistema	7	5,0%
Fazer login no sistema	7	5,0%
Total	**141**	**100,0%**

Fonte: Elaboração própria, com base em dados do suporte (2026).

A categoria "Outro" agrega solicitações heterogêneas que não foram classificadas em uma categoria específica no levantamento original, incluindo pedidos de alteração de categoria do especialista. Ainda que não tenha sido possível isolar cada uma dessas demandas na catalogação, observa-se que quase metade dos chamados (48,2%) corresponde a ajuste cadastral ou de

pontuação, o que evidencia que o volume de trabalho manual não decorria apenas de casos isolados, mas de operações recorrentes de manutenção de dados.

A execução manual de comandos SQL em ambiente de produção, para atender a esse tipo de demanda, apresentava riscos que não se restringiam ao volume de chamados:

- **Risco de inconsistência:** execução manual sujeita a erro humano, sem validação automática das regras de negócio;
- **Ausência de rastreabilidade:** falta de registro sobre quem solicitou, quem aprovou e qual era o estado anterior do dado alterado;
- **Ineficiência operacional:** alocação de esforço técnico em tarefas repetitivas de suporte administrativo;
- **Dependência da equipe de TI:** indisponibilidade do responsável com acesso ao banco interrompia o atendimento.

O diagnóstico evidenciou a necessidade de substituir o fluxo manual por um módulo integrado ao sistema, capaz de formalizar as solicitações, submetê-las a uma máquina de estados para aprovação e rejeição (descrita na Seção **??**) e manter histórico auditável das alterações. O módulo foi concebido para absorver três tipos de demanda identificados no levantamento: alteração de categoria, cadastro de pontuação e complemento cadastral. A alteração de categoria, por envolver dados do usuário e exigir possibilidade de reversão, recebeu tratamento específico, com registro de snapshot e linha do tempo, conforme detalhado na Seção 3.3.

3.1.5 Levantamento de Requisitos

O levantamento de requisitos teve como objetivo definir as funcionalidades e as propriedades de qualidade necessárias à integração do módulo de solicitações ao sistema Banco de Especialistas. A solução foi concebida para preservar o comportamento das funcionalidades existentes e formalizar o processo de alteração de categoria e de dados cadastrais dos especialistas, mediante validação das entradas, controle de aprovação e registro das operações realizadas.

Os requisitos funcionais especificam as operações disponibilizadas pelo módulo e seus efeitos sobre os registros do sistema. Os requisitos não funcionais estabelecem as condições de integridade, segurança, rastreabilidade, compatibilidade e manutenção que orientam a implementação da solução.

3.1.5.1 Requisitos Funcionais

Os requisitos funcionais abrangem a criação e o processamento de solicitações, a consulta e o acompanhamento das demandas, a aplicação das alterações aprovadas, a preservação do estado anterior dos registros e o controle de acesso às operações. As decisões e as alterações devem ser executadas exclusivamente por usuários autorizados, conforme as regras de negócio estabelecidas.

Quadro 1 – Requisitos funcionais do módulo de solicitações.

ID	Descrição	Tipo	Origem	Impl.
RF01	Permitir a criação de solicitações de mudança de categoria, com categoria de destino e justificativa obrigatória.	Categoria	Seção 3.1.4	Seção 3.2
RF02	Permitir a criação de solicitações de atualização de pontuação, com indicação dos itens e das comprovações.	Pontuação	Seção 3.1.4	Seção 3.2
RF03	Permitir a criação de solicitações de complemento cadastral ou recurso, com descrição do problema identificado.	Complemento	Seção 3.1.4	Seção 3.2
RF04	Permitir a consulta das solicitações, com filtros por tipo e situação, conforme o perfil do usuário.	Todos	Seção 3.1.4	Seção 3.4
RF05	Permitir a aprovação ou rejeição de solicitações, registrando o responsável e a decisão tomada.	Todos	Seção 3.1.4	Seção 3.2
RF06	Aplicar, após a aprovação, as alterações correspondentes à categoria de destino do especialista.	Categoria	Seção 3.1.3	Seção 3.2
RF07	Aplicar, após a aprovação, a atualização da pontuação do especialista correspondente aos itens aprovados.	Pontuação	Seção 3.1.3	Seção 3.2
RF08	Registrar o estado anterior do especialista antes da aplicação de alterações em snapshot.	Categoria	Seção 3.1.4	Seção 3.3
RF09	Permitir a reversão de alterações aprovadas, restaurando o estado anterior registrado.	Categoria	Seção 3.1.4	Seção 3.3
RF10	Registrar e disponibilizar uma linha do tempo cronológica dos eventos associados a cada solicitação.	Todos	Seção 3.1.3	Seção 3.3
RF11	Restringir a criação, a consulta e a decisão a usuários autenticados e autorizados conforme o perfil.	Todos	Seção 3.1.4	Seção 3.2

Fonte: Elaboração própria (2026). **Nota:** O requisito RF03 refere-se à abertura de solicitações de complemento cadastral ou recurso. Após a aprovação ou rejeição, o sistema registra a decisão e o encaminhamento correspondente, sem alterar automaticamente o perfil do especialista.

3.1.5.2 Requisitos Não Funcionais

Os requisitos não funcionais estabelecem as propriedades de qualidade necessárias ao funcionamento do módulo e à sua integração ao sistema existente. Esses requisitos abrangem a integridade das operações, a rastreabilidade das alterações, a segurança do controle de acesso, a validação dos dados, a compatibilidade com as funcionalidades existentes, a extensibilidade da solução, a testabilidade das regras de negócio e a reprodutibilidade do ambiente de testes.

Quadro 2 – Requisitos não funcionais do módulo de solicitações.

ID	Categoria	Descrição	Mecanismo
RNF01	Integridade	As alterações decorrentes da aprovação devem ser executadas atomicamente, evitando persistência parcial dos registros envolvidos.	Transações PostgreSQL e SQLAlchemy
RNF02	Rastreabilidade	As informações de auditoria devem permitir reconstituir o histórico de decisões e o estado anterior registrado em snapshot.	Entidade Snapshot e linha do tempo de eventos
RNF03	Segurança	O controle de acesso deve ser aplicado no servidor para impedir operações não autorizadas por chamadas diretas à API.	Verificação de permissões nas rotas
RNF04	Validação	Os dados recebidos pela API devem ser validados no servidor, tornando a validação do backend autoritativa.	Schemas Marshmallow
RNF05	Compatibilidade	A incorporação do módulo deve preservar o comportamento das funcionalidades existentes não abrangidas pela alteração.	Blueprints Flask e mudança aditiva
RNF06	Extensibilidade	A inclusão de novos tipos de solicitação deve permitir reaproveitamento do fluxo principal e ajustes mínimos nas regras específicas.	Formulário único e configuração `SCHEMA_POR_TIPO`
RNF07	Testabilidade	As regras de negócio devem permitir testes isolados sem requisição HTTP, verificando a camada de serviço independentemente.	Separação da camada de serviço e Pytest
RNF08	Reprodutibilidade	O ambiente de testes deve ser configurado e executado de maneira reproduzível, com dependências e procedimentos claros.	Docker e Docker Compose

Fonte: Elaboração própria (2026).

3.2 Desenvolvimento do Módulo de Solicitações

3.2.1 Endpoints

A camada de rotas do módulo de solicitações foi implementada como um *Blueprint* do Flask, registrado sob o prefixo `/solicitations`. A escolha do *Blueprint* decorre diretamente

da estratégia de *Additive Change* descrita na Seção **??**: o módulo é incorporado como um componente isolado, sem alterar os controladores existentes. As rotas atuam como camada de fronteira — recebem a requisição, aplicam o controle de acesso, delegam o processamento ao `SolicitationService` e serializam a resposta —, mantendo fora do controlador as regras de negócio, a persistência e o tratamento de exceções de domínio. A documentação Swagger foi externalizada em arquivos YAML, referenciados pelo decorador `@swag_from`, em contraste com o padrão observado nos controladores legados (Algoritmo 1), nos quais a documentação ocupava a maior parte da função.

O Quadro 3 relaciona os endpoints implementados, a função responsável por cada um, a restrição de acesso aplicada e os requisitos funcionais atendidos.

Quadro 3 – Endpoints implementados nos módulos de solicitações e revisão.

Método	**Rota**	**Controle de acesso**	**Requisitos**
POST	`/`	Usuário autenticado	RF01, RF02, RF03
GET	`/`	Autenticado com perfil de visualização	RF04
GET	`/me`	Usuário autenticado (restrito ao próprio usuário)	RF04
GET	`/<id>`	Autenticado, com verificação de acesso no serviço	RF04, RF11
POST	`/review`	Usuário autenticado com `SPECIALIST_REVIEW`	RF05, RF06, RF07
POST	`/review/edit`	Usuário autenticado com `SPECIALIST_REVIEW`	RF05, RF06, RF07
GET	`/review/<solicitation_id>`	Usuário autenticado com `SPECIALIST_REVIEW`	RF05, RF06, RF07
GET	`/review/user/<user_id>`	Usuário autenticado com `SPECIALIST_REVIEW`	RF05, RF06, RF07

Fonte: Elaboração própria (2026). Obs.: as rotas do módulo de solicitações são prefixadas com `/solicitations`, enquanto as rotas de revisão utilizam o prefixo `/solicitation/review`.

A rota de criação (`POST /solicitations`) concentra o padrão adotado nas demais. O controlador limita-se a extrair os dados da requisição — aceitando tanto `multipart/form-data`, quando há arquivo anexado, quanto JSON, quando não há — e a delegar o processamento ao serviço, conforme ilustrado no Algoritmo 3.

Quatro características do padrão merecem destaque. A primeira é o **controlador fino**: a função não constrói entidades, não valida campos individualmente, não executa consultas e não trata exceções de banco — responsabilidades que, no padrão legado (Algoritmo 1), estavam concentradas na própria rota. A segunda é o **controle de acesso declarativo**, aplicado pelo decorador `@auth.login_required`, que aceita um parâmetro opcional de perfil; a rota de listagem geral exige perfil de visualização, enquanto a rota `/me` restringe o resultado ao próprio usuário autenticado, independentemente do perfil. A terceira é a **validação na fronteira** por schemas Marshmallow: a consulta é interpretada por `SolicitationQuerySchema`, e a resposta é serializada por `SolicitationSchema` ou `PaginatedResponseSchema`, de modo

Algoritmo 3: Estrutura da rota de criação de solicitação.

```
@bp.route("", methods=["POST"])
@auth.login_required
@swag_from("docs/create_solicitation.yml", validation=False)
def save_solicitation():
    form_data = request.form.to_dict() if request.form else None
    json_data = request.get_json(silent=True) or {}
    file = request.files.get("document_file")

    return SolicitationService.create_or_update(
        user_id=g.user.id,
        form_data=form_data,
        file=file,
    )
```

que o formato de entrada e o de saída não dependem da interface que consome a API, atendendo ao RNF04. A quarta é a **externalização da documentação**: o decorador `@swag_from` referencia um arquivo YAML por rota, com `validation=False` indicando que a validação é responsabilidade dos schemas e não do Swagger.

A listagem foi implementada por meio de uma função auxiliar, `_fetch_solicitations`, que centraliza a interpretação dos parâmetros de paginação, ordenação e filtro (situação, intervalo de datas e usuário). O endpoint `/solicitations/me` reutiliza a mesma função forçando o identificador do usuário autenticado, enquanto o endpoint `/solicitations` permite filtro opcional por usuário, desde que o solicitante possua o perfil de visualização. Essa centralização evita a duplicação da lógica de consulta entre rotas, que, no padrão legado, costumava ser reproduzida em cada controlador.

Do ponto de vista dos requisitos, os endpoints concentram os RF01 a RF04 (criação e consulta) e o RF11 (restrição de operações a usuários autorizados). As operações de decisão, aplicação de efeitos, snapshot e reversão, correspondentes aos RF05 a RF09, são implementadas por rotas específicas descritas na Seção 3.2.

3.2.2 Integração com o Banco de Dados

A persistência do módulo foi implementada com SQLAlchemy sobre PostgreSQL, seguindo o padrão já adotado pelo sistema. O mapeamento foi feito em estilo declarativo, com uma tabela principal (`solicitations`) e duas tabelas de referência (`solicitation_status` e `solicitation_category`), que funcionam como *lookup tables* para os estados e os tipos de solicitação. A escolha de manter tabelas de referência, em vez de armazenar os valores como texto livre, garante integridade referencial no banco e permite a evolução do domínio sem migrations estruturais a cada novo valor. O Quadro 4 descreve as tabelas do módulo, suas colunas e restrições.

Quadro 4 – Modelo de dados do módulo de solicitações.

Tabela	**Coluna**	**Tipo / Restrição**	**Observação**
`solicitations`	`id`	Inteiro, PK	Identificador da solicitação
	`id_user`	FK → `users`, obrigatório	Solicitante
	`id_status`	FK → `solicitation_status`, padrão `CREATED`	Estado atual
	`id_category`	FK → `solicitation_category`, opcional	Tipo de solicitação; nulo em rascunho
	`specialist_type_id`	FK → `users_specialist_types`, opcional	Categoria de destino
	`title`	Texto(128), obrigatório	Título
	`description`	Texto(640), obrigatório	Justificativa ou descrição
	`file_path`	Texto(256), opcional	Caminho do documento anexado
	`created_at`	DataHora, padrão `agora()`	Criação
	`last_update`	DataHora, atualizado em alteração	Última modificação
`solicitation status`	`id`, `name`	PK, Texto(20) único	`CREATED`, `PENDING`, `APPROVED`, `REJECTED`
`solicitation category`	`id`, `name`	PK, Texto(20) único	`CHANGE_CATEGORY`, `RESOURCE`, `UPDATE_POINTS`
`solicitation reviews`	`id_solicitation`	FK → `solicitations`, cascata	Decisões e pareceres das análises

Fonte: Elaboração própria (2026).

O Algoritmo 4 descreve, em pseudocódigo, a estrutura da entidade principal, com seus atributos, restrições, índices e relacionamentos, e as enumerações que definem os estados e os tipos de solicitação.

Algoritmo 4: Estrutura da entidade `Solicitation` e enumerações de domínio (pseudocódigo).

```
ENUMERAÇÃO SolicitationStatus
    CREATED  = 1    // rascunho, em elaboração
    PENDING  = 2    // submetida, aguardando análise
    APPROVED = 3    // deferida
    REJECTED = 4    // indeferida

ENUMERAÇÃO SolicitationCategory
    CHANGE_CATEGORY = 1    // alteração de categoria
    RESOURCE        = 2    // recurso / complemento cadastral
    UPDATE_POINTS   = 3    // atualização de pontuação

ENTIDADE Solicitation
    TABELA: solicitations

    ATRIBUTOS
        id                 : Inteiro, Chave Primária
        id_user            : Inteiro, FK -> User, obrigatório
        id_status          : Inteiro, FK -> SolicitationStatus,
                             padrão CREATED
        id_category        : Inteiro, FK -> SolicitationCategory,
                             opcional (discriminador de tipo)
        specialist_type_id : Inteiro, FK -> UserSpecialistType,
                             opcional (categoria de destino)
        title              : Texto(128), obrigatório
        description        : Texto(640), obrigatório
        file_path          : Texto(256), opcional
        created_at         : DataHora, padrão agora()
        last_update        : DataHora, atualizado em cada alteração

    RELACIONAMENTOS
        user    : User                 (muitos-para-um)
        reviews : Lista<Review>        (um-para-muitos, cascata)

    ÍNDICES
        idx_solicitation_user     -> (id_user)
        idx_solicitation_status   -> (id_status)
        idx_solicitation_created  -> (created_at)
```

A decisão de modelagem central é a **abordagem polimórfica em tabela única**: os três tipos de solicitação (`CHANGE_CATEGORY`, `RESOURCE` e `UPDATE_POINTS`) compartilham a

mesma tabela `solicitations`, discriminados pelo campo `id_category`. Essa escolha evita a fragmentação em três tabelas com colunas comuns replicadas (título, descrição, autor, estado, timestamps) e concentra a lógica de listagem, filtragem e paginação em uma única consulta, independentemente do tipo. Campos específicos de um tipo, como `specialist_type_id`, são opcionais e populados apenas quando aplicáveis: no fluxo de alteração de categoria, o campo indica a categoria de destino; nos fluxos de recurso e de atualização de pontuação, permanece nulo. A abordagem foi adotada em vez da herança de tabela por subtipo (*joined table inheritance*) por não haver, no escopo do módulo, atributos exclusivos de um tipo que justificassem tabelas próprias.

O **estado de rascunho** é consequência direta dessa modelagem. O estado `CREATED` representa uma solicitação em elaboração, que pode ser salva e editada antes da submissão. Nesse estado, o campo `id_category` pode ser nulo, permitindo que o usuário salve a solicitação parcialmente preenchida e escolha o tipo mais adiante. A submissão para análise corresponde à transição de `CREATED` para `PENDING`, momento a partir do qual o tipo passa a ser obrigatório na prática, ainda que a restrição não esteja expressa no esquema. A alternativa — tornar `id_category` obrigatório e criar uma tabela separada para rascunhos — foi descartada por duplicar a estrutura e a lógica de consulta sem ganho proporcional.

A **dupla representação de estados e categorias** — enumeração em Python e tabela de referência no banco — atende a duas necessidades distintas. A enumeração serve à aplicação, garantindo, em tempo de desenvolvimento, que apenas valores conhecidos sejam usados e permitindo comparações explícitas no código. A tabela de referência serve ao banco, garantindo integridade referencial e viabilizando junções e filtros eficientes nas consultas de listagem. O custo é a disciplina de manter os dois em sincronia a cada novo valor; o benefício é que o estado de uma solicitação nunca fica armazenado como texto livre, o que facilitaria inconsistências e inviabilizaria restrições declarativas.

A **chave estrangeira para** `users`, e não para a entidade de especialista, reflete no esquema o desafio identificado na Seção 3.1.3: a categoria do especialista está em `users`, e não em `specialists`. A solicitação referencia o usuário e, quando aplicável, indica a categoria de destino em `specialist_type_id`, sem duplicar a classificação corrente. Isso mantém o modelo coerente com o restante do sistema e evita que a solicitação armazene estados intermediários que precisariam ser sincronizados com a tabela de origem.

A **indexação** foi definida a partir das consultas efetivamente implementadas nas rotas (Seção 3.2.1): listagem por usuário (`idx_solicitation_user`), listagem por situação (`idx_solicitation_status`) e ordenação por data de criação (`idx_solicitation_created`). Consultas por identificador já são otimizadas pela chave primária e não exigem índice adicional. A escolha privilegia os caminhos de leitura da interface — listagem do solicitante, listagem do avaliador e ordenação cronológica — em vez de uma indexação genérica e não fundamentada.

O **armazenamento do arquivo** foi delegado ao sistema de arquivos: o atributo `file_path` guarda o caminho do documento anexado, não o conteúdo binário. Essa decisão

mantém o banco fora do caminho de operações de I/O de arquivos, que são executadas fora da transação principal, e evita o inchaço da tabela. A referência ao arquivo permanece no mesmo registro da solicitação, mantendo a consistência entre o documento e os metadados que o descrevem.

Por fim, o **controle temporal** foi delegado ao banco: `created_at` recebe o instante corrente na inserção e `last_update` é atualizado automaticamente a cada alteração, de modo que o momento da última modificação é registrado mesmo em operações que não passem pela aplicação. Isso atende parcialmente ao RNF02, no aspecto temporal da rastreabilidade; a dimensão semântica — quem alterou, de qual estado para qual estado, com qual justificativa — é atendida pelo mecanismo de snapshot e pela linha do tempo, descritos na Seção 3.3. A atomicidade das operações (RNF01), por sua vez, não é garantida no modelo, mas na camada de serviço, por meio da unidade de trabalho do SQLAlchemy (`session`): o modelo define as restrições; a transação define o momento em que elas são verificadas em conjunto.

3.2.3 Autorização e Controle de Acesso

A autorização foi implementada considerando os diferentes papéis envolvidos no fluxo, de modo que apenas usuários com permissão adequada possam aprovar, rejeitar ou alterar as informações relacionadas à categoria do especialista. Esse controle é essencial para preservar as regras de negócio e evitar ações indevidas no sistema.

3.2.4 O Módulo de Revisão e a Efetivação das Alterações

A organização do fluxo de solicitações foi concebida para separar estritamente a intenção (o pedido do usuário) da efetivação (a decisão e o impacto no cadastro). Para tornar essa arquitetura compreensível e manter a coesão do relatório, esta seção divide-se em três aspectos: a separação de responsabilidades e controle de acesso, a modelagem da revisão como evento de domínio e, por fim, a execução das regras de negócio e a garantia de atomicidade transacional.

3.2.4.1 Separação de Responsabilidades e Controle de Acesso

O módulo de *Solicitation* atua exclusivamente como o registro da intenção, enquanto o módulo de *Review* (Revisão) assume a responsabilidade pela validação, pela decisão e pela efetivação do efeito prático sobre o especialista. A criação da solicitação apenas documenta o pedido; a decisão autorizada e o impacto material no cadastro ocorrem no controlador *create_review*.

Esse componente exige a permissão RBAC `SPECIALIST_REVIEW`, restringindo a execução da aprovação ou rejeição a usuários com competência formal para atuar sobre o perfil do especialista. A estrutura adotada atende ao requisito de controle de acesso do sistema e evita que alterações de categoria ou de pontuação sejam aplicadas por atores sem a devida autorização.

3.2.4.2 A Revisão como um Evento de Domínio (Domain Event)

Para assegurar que a decisão seja um registro histórico auditável, a entidade `SolicitationReview` foi modelada para atuar como um **Evento de Domínio** (*Domain Event*), segundo os preceitos do *Domain-Driven Design* (EVANS, 2003). Em vez de ser apenas uma tabela de passagem, cada registro na tabela `solicitation_reviews` representa o fato imutável de que uma decisão ocorreu no domínio (o evento de aprovação ou rejeição).

A modelagem armazena a chave estrangeira para a solicitação (`id_solicitation`), o avaliador responsável (`id_user`), o veredicto (`verdict`), a justificativa obrigatória (`comment`) e os carimbos temporais (`created_at` e `last_update`). A inclusão do usuário garante a dimensão autoral da auditoria, enquanto o campo `id_category` mantém a rastreabilidade semântica. Adicionalmente, foram definidos índices específicos (`idx_review_solicitation`, `idx_review_user` e `idx_review_created`) para otimizar a reconstituição da linha do tempo. Ao tratar a revisão como um Evento de Domínio, o sistema materializa o gatilho que dispara os efeitos transacionais no especialista, atendendo diretamente ao RNF02 (Rastreabilidade).

3.2.4.3 Execução das Regras de Negócio e Atomicidade

Antes da efetivação da decisão, o processo de revisão executa validações de consistência e conflito para garantir que a operação seja compatível com o estado atual do especialista. O sistema impede, por exemplo, a mudança para a mesma categoria em que o especialista já se encontra e bloqueia aprovações quando o profissional já está em processo de retificação. Essas verificações funcionam como guardas de negócio: a decisão só é aceita quando o contexto do especialista atende às condições mínimas de validade. Em termos de arquitetura, essa etapa funciona como camada de validação antes da aplicação dos efeitos, garantindo que os requisitos **RF05**, **RF06** e **RF07** sejam atendidos de maneira consistente.

Quando a revisão é aprovada (*verdict=True*), a lógica de negócio é delegada à função interna *_apply_review_business_rules(solicitation)*, que atua como uma máquina de estados associada ao tipo da solicitação. Para solicitações do tipo `CHANGE_CATEGORY`, o fluxo aplica a mudança de classificação por meio de *SpecialistService.switch_category* e, em seguida, dispara o processo de retificação via *rectify_specialist_service*. Já para solicitações do tipo `UPDATE_POINTS`, o sistema não altera a classificação, mas executa apenas a retificação do perfil, recalculando a pontuação conforme os critérios definidos.

A etapa de retificação constitui o ponto crítico de atomicidade do processo. A função *rectify_specialist_service* recebe o parâmetro *commit=False*, o que significa que ela calcula a nova pontuação, altera o status do especialista para `RECTIFYING`, registra a ação na linha do tempo e gera a notificação para o usuário, mas não confirma a transação em si. A confirmação é realizada na camada superior, por meio de *flush* ou do fechamento explícito da transação, garantindo que todas as alterações sejam persistidas em conjunto. Esse mecanismo atende ao requisito **RNF01**, pois assegura a consistência do estado do especialista e a impossibilidade

de persistir apenas parte das alterações em caso de falha operacional. Além disso, o uso de *commit=False* preserva a auditoria do sistema: a cronologia de ações, a notificação enviada ao usuário e a alteração de status ficam vinculadas ao mesmo evento decisório, permitindo que a responsabilidade pela mudança seja rastreada de forma inequívoca.

3.3 Implementação do Sistema de Snapshot e Auditoria

3.3.1 Entidade Snapshot e Modelo de Persistência

A entidade `SpecialistSnapshot` foi criada para preservar o estado do especialista antes da aplicação de alterações aprovadas no fluxo de mudança de categoria. O nome reflete o escopo: o snapshot cobre o especialista, e é acionado exclusivamente pelo tipo `CHANGE_CATEGORY` — recurso e atualização de pontuação não geram snapshot, pelas razões discutidas na Seção **??**. O Quadro 5 descreve sua estrutura.

Quadro 5 – Modelo da entidade `SpecialistSnapshot`.

Coluna	**Tipo**	**Restrição**	**Observação**
`id`	Inteiro	Chave primária	Identificador do snapshot
`specialist_id`	Inteiro	Obrigatório, indexado	Especialista cujo estado foi capturado; sem FK
`snapshot`	JSONB	Obrigatório	Estado anterior serializado
`authorizer_id`	Inteiro	Obrigatório	Usuário que autorizou a alteração; sem FK
`created_at`	DataHora	Padrão `agora()`	Momento da captura

Fonte: Elaboração própria (2026).

Duas decisões de modelagem merecem destaque. A primeira é a **ausência de chave estrangeira** em `specialist_id` e `authorizer_id`. Trata-se de uma escolha deliberada para uma entidade de auditoria: o snapshot é um registro histórico, e sua integridade não deve depender da existência atual da entidade referenciada. Se o especialista ou o autor da decisão for removido do sistema, o snapshot permanece consultável, preservando a memória do que ocorreu. A contrapartida é a possibilidade de referências órfãs e a ausência de integridade referencial garantida pelo banco — custo aceito em troca da preservação histórica. A alternativa — FK com `ON DELETE SET NULL` — destruiria justamente a informação que o snapshot existe para preservar.

A segunda é a **ausência de vínculo explícito com a solicitação** que originou o snapshot. A entidade referencia o especialista, não a solicitação nem a revisão. Essa decisão tem consequência direta sobre a reversão (Seção 3.3.2): a operação de reversão busca o snapshot mais recente do especialista, e não o snapshot "daquela solicitação". Para o fluxo atual — em que se reverte a última alteração aprovada —, a modelagem é suficiente. Para reversão de uma alteração específica no meio do histórico, seria necessário vincular o snapshot à solicitação ou à revisão correspondente, o que está registrado como limitação em Seção 3.3.4.

O campo `snapshot` armazena o estado anterior em JSONB. A escolha do formato, em vez de colunas relacionais, decorre de três fatores. O primeiro é a **coesão do estado**: os campos capturados descrevem um único estado lógico do especialista e fazem sentido juntos; armazená-los em uma coluna única preserva essa coesão e evita reconstituição por junção. O segundo é a **evolução do esquema**: se o conjunto de campos a capturar for ampliado, um documento JSONB acomoda novos campos sem migration; colunas adicionais exigiriam alteração de esquema e migração de dados históricos. O terceiro é a **legibilidade**: consultas diretas ao documento permitem visualizar o estado anterior sem conhecer o esquema das tabelas de origem.

As contrapartidas também precisam ser declaradas. O JSONB não impõe integridade referencial interna — o documento é um retrato, não uma referência viva, e não há como garantir que os campos correspondam ao esquema atual do especialista depois de migrations futuras. O formato não é validado pelo banco: a estrutura do documento depende da aplicação que o grava. Essas limitações são aceitáveis no escopo do módulo, porque o snapshot é um registro histórico e **deve** preservar o estado como era no momento da captura, e não acompanhar a evolução do esquema. A alternativa — colunas relacionais com FK para as tabelas de origem — traria integridade, mas produziria snapshots que se corrompem quando a tabela de origem muda, anulando o propósito de preservar o estado anterior.

A captura é feita antes da aplicação dos efeitos e dentro da mesma unidade de trabalho da aprovação (Seção **??**): o snapshot, a mudança de categoria e o registro da revisão são confirmados em conjunto. Se qualquer etapa falhar, o snapshot é descartado junto, garantindo que ele exista se e somente se a alteração correspondente foi efetivada. Isso atende ao RNF01 e sustenta o RNF02: sem atomicidade, um snapshot poderia registrar um estado anterior que nunca foi alterado, ou uma alteração sem snapshot correspondente.

O campo `authorizer_id` registra o usuário responsável pela autorização da alteração, separadamente do autor da revisão. Na prática, os dois coincidem no fluxo atual — o avaliador que aprova a revisão é o mesmo que autoriza a alteração —, mas manter o campo explícito na entidade permite que, no futuro, a autorização seja delegada ou automatizada sem alteração do modelo. A separação também garante que a informação esteja presente na própria entidade de auditoria, e não apenas derivável por junção com a revisão.

3.3.2 Reversão de Alterações

A reversão permite desfazer uma alteração de categoria aprovada, restaurando o estado do especialista registrado no snapshot mais recente. A operação é acionada por rota administrativa e está sujeita a controle de acesso, como as demais operações de decisão (RNF03).

A reversão não é uma operação destrutiva: o snapshot não é apagado após o uso, e a reversão é registrada como evento próprio na linha do tempo (Seção 3.3.3). O histórico completo é preservado — a solicitação original, a revisão que aprovou, o snapshot capturado,

a reversão e o estado resultante. Essa é a diferença entre reversão e exclusão de histórico: a primeira adiciona um evento, a segunda remove o passado. A escolha pela primeira é o que sustenta a auditoria.

O Algoritmo 5 descreve o fluxo em pseudocódigo, com base na recuperação do snapshot mais recente do especialista.

Algoritmo 5: Fluxo de reversão (pseudocódigo).

```
FUNÇÃO reverter(especialista_id, autorizador):
    especialista = buscar Specialist por id
    SE não existe:
        erro "especialista não encontrado"

    snapshot = buscar SpecialistSnapshot
               filtrando specialist_id = especialista.id
               ordenando por created_at DESC
               limitando a 1 resultado
    SE não existe:
        erro "nenhum snapshot encontrado"

    estado_anterior = desserializar(snapshot.snapshot)

    EM UMA TRANSAÇÃO:
        restaurar(especialista, estado_anterior)
        registrar_evento(especialista, REVERTIDA,
                         autorizador, data)

    RETORNAR confirmação
```

Quatro decisões delimitam o escopo da reversão. A primeira é que ela restaura o **snapshot mais recente**, não o snapshot de uma solicitação específica. Isso decorre da modelagem da entidade (Seção 3.3.1), que referencia o especialista sem vincular-se à solicitação. A consequência é que a reversão desfaz a última alteração aprovada, e não uma alteração qualquer no histórico. Para o caso de uso original — corrigir uma mudança de categoria recente que se mostrou inadequada —, a semântica é adequada. Para reverter uma alteração específica no meio do histórico, seria necessário alterar o modelo para vincular o snapshot à solicitação. A limitação está registrada em Seção 3.3.4.

A segunda é que a reversão restaura apenas os campos capturados no snapshot, sem desfazer alterações feitas ao especialista por outros fluxos no intervalo. Se, após a aprovação, o especialista atualizou dados pessoais ou pontuação por outro caminho, esses dados não são afetados. A escolha é conservadora e evita que uma reversão de categoria apague, por tabela, alterações não relacionadas.

A terceira é que a reversão **não altera o estado da revisão** que aprovou a solicitação. A revisão permanece registrada como aprovada, e a reversão é um evento subsequente — o

que preserva o histórico de decisões.

A quarta é que a operação adiciona um novo evento à linha do tempo, mas não gera um novo snapshot. Isso poderia ser questionado — se a reversão também é uma alteração, por que não capturar o estado anterior a ela? A resposta é que reverter uma reversão seria equivalente a reaplicar a alteração original, e isso é feito criando-se uma nova solicitação, não reusando o snapshot antigo. A decisão mantém o modelo simples: um snapshot por alteração aprovada, um evento de reversão por reversão executada.

3.3.3 Linha do Tempo de Eventos

A linha do tempo é a apresentação cronológica dos eventos associados a um especialista: criação da solicitação, submissão para análise, decisão da revisão, captura de snapshot e, quando aplicável, reversão. A implementação **não mantém tabela própria de eventos**: a linha do tempo é derivada das entidades que já registram cada evento com autor e data — `Solicitation` (`created_at`), `SolicitationReview` (autor, veredicto, comentário, data) e `SpecialistSnapshot` (autorização, captura, data). A escolha por derivação, em vez de uma tabela de log genérica, evita duplicação de informação e garante que a linha do tempo nunca divirja das entidades de origem: se a revisão registrou uma decisão, ela aparece na linha do tempo; se o snapshot foi capturado, ele também aparece.

O preço da derivação é que a consulta precisa agregar três fontes e ordená-las por data. Isso é feito no serviço de consulta da linha do tempo, que retorna a lista consolidada para a interface. A alternativa — tabela de eventos dedicada, alimentada por gatilhos — teria a vantagem de uma consulta única, mas exigiria sincronização com as entidades de origem e abriria margem para inconsistência. Para o volume atual, a derivação é a solução mais simples e mais robusta.

A linha do tempo cumpre duas funções. Para o solicitante, mostra o andamento da solicitação e a justificativa de cada decisão. Para o avaliador e para auditoria, permite reconstituir o processo completo, com autores, datas e estados intermediários. Em ambos os casos, a informação apresentada é a mesma que sustenta a rastreabilidade — a interface apenas a torna legível.

3.3.4 Rastreabilidade e Limitações

O conjunto formado por solicitação, revisão, snapshot e linha do tempo atende ao requisito de rastreabilidade (RNF02) em três dimensões. A dimensão **temporal** é coberta pelos carimbos automáticos das entidades. A dimensão **autoral** é coberta pelo registro do solicitante na solicitação, do avaliador na revisão e do autorizador no snapshot. A dimensão **semântica** — o que mudou e por quê — é coberta pelo snapshot (estado anterior) e pela revisão (justificativa). As três juntas permitem reconstituir, para qualquer solicitação, quem pediu, quem decidiu, quando, com qual justificativa e de qual estado para qual estado.

O mecanismo tem limitações que precisam ser declaradas. A primeira é a já discutida **reversão pelo snapshot mais recente**: como o snapshot não referencia a solicitação, a reversão desfaz a última alteração aprovada, e não uma alteração específica. A segunda é a **ausência de exclusão lógica**: o `cascade="all, delete-orphan"` na relação entre solicitação e revisões implica que a remoção física de uma solicitação apaga o histórico associado. O módulo não expõe rota de exclusão, de modo que o cascade atua apenas como salvaguarda de integridade, mas a ausência de uma política de retenção é uma lacuna que pode ser endereçada em trabalho futuro. A terceira é a **ausência de vínculo do snapshot com a solicitação**: o snapshot referencia o especialista, não a solicitação, o que simplifica a modelagem e atende ao caso de uso atual, mas impede auditoria retroativa que relacione snapshot e revisão diretamente — a associação existe apenas por proximidade temporal e autoral. A quarta é que **alterações fora do fluxo formal não geram snapshot**: intervenções diretas no banco, ainda que possíveis para a equipe de suporte, não geram revisão nem snapshot, apenas atualizam `last_update`. A política de acesso ao banco em produção está fora do escopo do módulo, mas o efeito sobre a rastreabilidade precisa ser reconhecido.

Essas limitações não invalidam o mecanismo; delimitam o que ele garante. O sistema passou de um fluxo sem registro formal (Seção 3.1.4) para um fluxo em que toda operação realizada pela aplicação é registrada com autor, data, justificativa e estado anterior. O que está fora do fluxo formal — intervenções diretas no banco — permanece fora do escopo da garantia, e isso é uma decisão de escopo, não uma falha do mecanismo.

3.4 Desenvolvimento das Interfaces Web

A camada de interface do módulo de solicitações foi desenvolvida em Nuxt.js sobre Vue.js e Vuetify, seguindo a arquitetura já adotada pelo Banco de Especialistas. As telas e componentes descritos nesta seção materializam, no lado do cliente, os requisitos funcionais de criação, consulta e decisão definidos na Seção 3.1.5, consumindo a API implementada em Flask (Seção 3.2) e respeitando o controle de acesso por perfil (RNF03). A organização em `pages/`, `components/`, `store/` e `layouts/` foi preservada, e o módulo novo foi incorporado como extensão das rotas e dos stores existentes, em coerência com a estratégia de *Additive Change* (Seção 3.1.2).

3.4.1 Telas Criadas

Foram implementadas três telas que cobrem o ciclo completo de uma solicitação: listagem administrativa, listagem do próprio solicitante e detalhe com revisão. As três compartilham o layout base da aplicação e a identidade visual do sistema, mas diferem em escopo, público e operações disponíveis.

3.4.1.1 Listagem administrativa — `pages/solicitations/index.vue`

A tela `pages/solicitations/index.vue` é o painel administrativo de solicitações. Ela atende ao RF04 (consulta com filtros) e é destinada a usuários com perfil de visualização, em conformidade com o Quadro 3 da Seção 3.2.1.

A tela carrega a lista via API no hook `created`, com paginação, ordenação e controle de itens por página. O estado local armazena `data` e `pagination` separadamente, o que permite atualizar a página sem recarregar o componente. Os campos exibidos são categoria, tipo do especialista, data de atualização e status, com CPF e tipo formatados para leitura. A tabela usa `v-data-table`, com chips de status coloridos por chave e navegação para o detalhe ao clicar na linha.

Do ponto de vista de projeto, a tela segue o padrão de composição do restante da aplicação: `template` para estrutura, `script` para carregamento, paginação e formatação, `style` para ajustes visuais e responsividade. O carregamento é tratado com indicador de *loading* e mensagem de erro em caso de falha da API, o que evita que o usuário interprete uma tabela vazia como ausência de solicitações.

3.4.1.2 Listagem do solicitante — `pages/solicitations/my_solicitation.vue`

A tela `pages/solicitations/my_solicitation.vue` é a visão do usuário final. Ela lista somente as solicitações do usuário autenticado, consumindo a rota `/solicitations/me` descrita na Seção 3.2.1, e restringe o resultado ao próprio solicitante independentemente do perfil — o que concretiza, na interface, a regra de escopo já garantida no servidor.

Diferentemente da listagem administrativa, que usa tabela, esta tela usa *cards* com dados resumidos (título, tipo, status e data) e navegação por número de páginas. A escolha é deliberada: o público-alvo é o solicitante, que precisa acompanhar poucos pedidos e reconhecê-los rapidamente; a tabela, mais densa, é adequada ao avaliador, que precisa comparar muitos registros.

A tela também concentra a regra de criação de novas solicitações. O botão "Criar Solicitação" é habilitado ou bloqueado conforme o retorno de `getDraftSolicitation()`: se a API responde 204 (sem rascunho em andamento), o botão é liberado; caso contrário, permanece bloqueado. Essa verificação espelha, no cliente, a regra de unicidade de solicitação em andamento descrita na Seção 3.1.3 e impede que o usuário inicie um segundo pedido antes de concluir o primeiro. A criação é feita em modal separado (`SolicitationModal`), o que mantém a listagem e o formulário desacoplados.

3.4.1.3 Detalhe e revisão — `pages/solicitations/_id.vue`

A tela `pages/solicitations/_id.vue` é a mais completa do fluxo e atende aos RF04, RF05, RF06 e RF07. Ela carrega a solicitação por identificador, monta um objeto `solicitation` a partir da resposta da API (título, descrição, categoria, solicitante, CPF, perfil

e anexo) e apresenta o painel de revisão com `v-form`, `v-textarea` e botões de aprovar e recusar.

A decisão é antecedida por três etapas. A primeira é a validação do formulário de revisão, que exige comentário e impede o envio em branco. A segunda é a confirmação explícita, exibida em diálogo que descreve o efeito da decisão — por exemplo, "a categoria do usuário será alterada de X para Y" —, o que reduz o risco de aprovação por engano em uma operação cujo efeito é sobre o cadastro do especialista. A terceira é o envio à API via `createReviewSolicitation`, com atualização imediata do estado local da interface em caso de sucesso.

O tratamento de erros é específico por regra de negócio, e não genérico. O método `handleSubmissionError` diferencia casos como `RectifySpecialistNotAllowedError` (especialista impedido de finalizar a ação), `SolicitationNotAllowedError` e `SolicitationNotFoundError` (redirecionamento para a lista do usuário). Essa distinção é relevante porque a API já retorna códigos semânticos (Seção 3.2.4.3); a interface limita-se a traduzi-los em mensagens e fluxos adequados, sem reimplementar as regras no cliente.

3.4.2 Componentes Vue

Os componentes reutilizáveis concentram a lógica de apresentação e de entrada de dados que não pertence a uma tela específica. No escopo do módulo, o principal é o `SolicitationModal.vue`, que encapsula o formulário de criação de solicitações.

O componente é um modal Vuetify com `v-form` e validação imediata, cobrindo título (máximo 128 caracteres), descrição (mínimo e máximo definidos por regra) e tipo da solicitação. Para o tipo "mudança de categoria", o campo "Nova Categoria" é exibido condicionalmente por meio de `v-if` e `transition`, refletindo no cliente a mesma lógica de esquema dinâmico que o backend implementa com `SCHEMA_POR_TIPO` (Algoritmo 2 da Seção 3.1.2). A correspondência entre os dois lados é intencional: a interface apenas antecipa, para conforto do usuário, uma restrição que o servidor já valida de forma autoritativa (RNF04).

O componente também trata do upload de documentos. Quando o tipo de solicitação exige comprovação, o envio é bloqueado se o arquivo estiver ausente ou exceder o limite de 2 MB. O envio é feito com `FormData`, o que permite transmitir campos textuais e o arquivo binário na mesma requisição, coerente com a rota `POST /solicitations` descrita na Seção 3.2.1, que aceita `multipart/form-data` quando há anexo. A validação no cliente é, novamente, antecipação da validação do servidor — não substitui a checagem no backend, que permanece autoritativa.

Além do `SolicitationModal`, o módulo reutiliza o `PreviewDocumentDialog`, empregado na tela de detalhe para visualizar o anexo antes da decisão. O componente é anterior ao módulo e foi reaproveitado sem alteração, o que evidencia a compatibilidade da estratégia aditiva: a interface nova consome componentes já consolidados, sem reescrevê-los.

No que diz respeito ao estado, as ações de acesso à API foram encapsuladas em módulos Vuex, em coerência com a organização existente. O módulo `store/specialists`

expõe `getSolicitations` e demais ações consumidas pelas telas; o módulo `store/appeal` isola as operações de recurso (buscar por usuário, criar e concluir apelação). A separação por domínio mantém as páginas focadas em apresentação e delega o acesso a dados ao store, o que reduz acoplamento e facilita a reutilização das mesmas ações em telas distintas.

3.4.3 Integração frontend/backend

A integração entre a interface e a API foi implementada com Axios, configurado no contexto do Nuxt. As telas não fazem chamadas HTTP diretas: elas despacham ações Vuex ou utilizam o cliente injetado (`this.$axios`), o que centraliza o tratamento de cabeçalhos, tokens e erros. Essa escolha é coerente com a arquitetura cliente-servidor e com o estilo REST descritos nas Seções 2.1.1 e 2.1.2, e é o que permite que a interface trate respostas de erro de forma padronizada.

O contrato entre os dois lados é o definido pelos schemas Marshmallow (Seção 2.3.5) e documentado em Swagger. A interface envia os campos esperados e consome os campos retornados, sem depender de estruturas internas do backend. O mapeamento explícito feito em `getDetailSolicitation` — que converte a resposta da API em um objeto `solicitation` com `createdBy`, `review` e `new_specialist_type` — ilustra essa separação: a forma da resposta pode evoluir sem que os componentes da interface precisem ser reescritos, bastando ajustar o mapeamento na tela.

O envio de arquivos usa `FormData`, como descrito em 3.4.2. O carregamento e o envio são acompanhados por indicadores visuais (`loading`, `loadingSolicitation`, `loading.approve`), e o retorno é apresentado com *snackbar* de sucesso ou de erro. O tratamento de erro distingue falhas de validação, de autorização e de regra de negócio, e redireciona o usuário quando a solicitação não existe ou não pertence a ele, o que evita que a interface permaneça em um estado inconsistente.

Cabe registrar que a paginação é tratada de forma explícita nas duas listagens. Em `index.vue`, o objeto `localSolicitations` mantém `data` e `pagination` separados, e a tela atualiza apenas a página corrente ao navegar. Em `my_solicitation.vue`, a navegação é feita por número de páginas, com o mesmo princípio. A consistência entre as duas telas foi deliberada: ambas consomem a mesma função auxiliar `_fetch_solicitations` no servidor (Seção 3.2.1), e manter o mesmo modelo de paginação no cliente simplifica a manutenção.

3.4.4 RBAC e controle de acesso

O controle de acesso foi implementado em duas camadas complementares. A camada autoritativa está no servidor, como descrito na Seção 3.2.3: as rotas verificam autenticação e perfil, e o serviço verifica se o solicitante tem acesso ao recurso. A camada de interface apenas reflete, para o usuário, o que o servidor já garante — não substitui a verificação no backend e não é tratada como mecanismo de segurança.

No cliente, o controle se manifesta de três formas. A primeira é a restrição de navegação: a listagem geral `index.vue` é acessível a usuários com perfil de visualização, enquanto `my_solicitation.vue` é a visão do solicitante, restrita por natureza ao próprio usuário. A segunda é a exibição condicional de operações: o botão de criação é habilitado somente quando a API confirma que não há rascunho em andamento, e o painel de revisão em `_id.vue` só é exibido a usuários com permissão de revisão. A terceira é o tratamento de erros de autorização: quando o servidor responde 403, a interface apresenta mensagem adequada e não insiste na operação.

Essa divisão de responsabilidades é coerente com o modelo RBAC descrito na Seção 2.5.3. O perfil `SPECIALIST_REVIEW` é exigido pelas rotas de revisão no servidor, e a interface apenas oculta o painel de decisão de quem não possui o perfil. A ocultação é conveniência de usabilidade, não controle de segurança: um usuário sem perfil que tente chamar a rota diretamente recebe 403 do servidor, o que preserva a integridade do fluxo independentemente da interface utilizada.

3.4.5 Síntese da seção

As interfaces web desenvolvidas cobrem o ciclo completo da solicitação: criação pelo solicitante, listagem administrativa, detalhe e decisão pelo avaliador. As três telas consomem a API implementada em Flask e respeitam o contrato definido pelos schemas, o que mantém a separação entre apresentação e regra de negócio. Os componentes reutilizáveis — `SolicitationModal` e `PreviewDocumentDialog` — e os módulos Vuex por domínio reduzem duplicação e preservam a organização existente. O controle de acesso é exercido de forma autoritativa no servidor e refletido na interface apenas como conveniência, em coerência com o modelo RBAC adotado.

Com isso, os requisitos RF04, RF05, RF06 e RF07 passam a ter correspondência direta nas telas descritas, e o RF11 (restrição de operações a usuários autorizados) é atendido pela combinação entre as verificações do servidor e a exibição condicional na interface. A Seção 3.5, que trata da verificação, avalia o comportamento dessas telas de forma integrada à API.

3.5 Testes e Validação

3.5.1 Construção da Infraestrutura de Testes

O sistema Banco de Especialistas não dispunha, antes do desenvolvimento deste módulo, de infraestrutura de testes automatizados. A verificação das funcionalidades era feita manualmente, por meio de requisições à API e inspeção do banco. Diante disso, a validação do módulo de solicitações exigiu a construção prévia de uma base de testes reutilizável, que pudesse ser aplicada não apenas ao módulo novo, mas a funcionalidades futuras e garantindo o funcionamentos das funcionalidades existentes.

A construção seguiu uma ordem deliberada. Primeiro, foi para funcionaldades básicas, que os novos módulos não deveria nem interferir, as features relacionadas a criação de especialistas,

tratamento de documentos, pontuação e RBAC. Em seguida, foram implementados testes de regressão para as funcionalidades existentes que o novo módulo afeta ou consome: timeline (`test_timeline.py`), usuário (`test_user.py`) e especialista (`test_specialist.py`). Esses testes estabeleceram uma linha de base — o comportamento que não poderia ser alterado. Só depois dessa linha de base, foram implementados os testes do próprio módulo de solicitações. A ordem não foi acidental: sem a linha de base, a refatoração introduzida pelo módulo novo não teria como ser verificada contra regressões.

No total, a suíte implementada contém 89 testes, distribuídos em 44 testes para o módulo novo e 45 testes de regressão. A proporção, próxima de 1:1, reflete a estratégia de equilíbrio entre adicionar cobertura para o que foi construído e preservar a cobertura do que existia. A suíte completa executa em aproximadamente três minutos, o que viabiliza sua integração à pipeline de verificação contínua.

A distribuição dos testes por arquivo é apresentada no Quadro 6.

Quadro 6 – Distribuição dos testes implementados.

Arquivo	**Testes**	**Propósito**
Módulo novo — fluxo completo		
`test_change_category.py`	19	Fluxo ponta a ponta: solicitação → revisão → mudança de categoria
Módulo novo — camadas		
`test_solicitation.py`	11	Integração pela API
`test_solicitation_service.py`	11	Camada de serviço
`test_review.py`	3	Integração pela API (decisão)
Regressão		
`test_notification.py`	22	Notificações — efeito colateral
`test_specialist.py`	8	Especialista — entidade modificada
`test_user.py`	7	Usuário — entidade associada
`test_stale_criteria_on_category_switch.py`	2	Critérios após mudança de categoria
`test_list_items_without_declaration.py`	2	Listagem de itens
`test_geral.py`	4	Verificações gerais
Total	**89**	

Fonte: Elaboração própria (2026).

O arquivo `test_change_category.py`, com 19 testes, é o mais relevante da suíte. Ele cobre o fluxo completo de mudança de categoria — criação da solicitação, aprovação pela rota de revisão, aplicação do efeito sobre o especialista e verificação do estado resultante —, para todas as transições válidas e para os casos de erro identificados na análise de regras de negócio (Seção 3.1.3). Diferentemente dos demais arquivos, que testam um módulo por vez, esses

testes atravessam os três módulos envolvidos no fluxo — solicitação, revisão e especialista —, e são a evidência prática de que a integração entre eles funciona. É a este arquivo que o restante deste capítulo se refere quando afirma que a mudança de categoria só existe como integração (Seção **??**).

3.5.2 Isolamento entre Testes

O isolamento entre testes é obtido por `TRUNCATE` das tabelas transacionais antes e depois de cada teste. A escolha do `TRUNCATE` com `RESTART IDENTITY CASCADE`, em vez do padrão mais comum de *rollback* transacional, foi deliberada e decorre de uma característica específica do ambiente.

O padrão mais difundido para isolamento em testes com SQLAlchemy é abrir uma transação por teste e fazer *rollback* ao final. Esse padrão funciona quando todas as operações do teste compartilham a mesma sessão de banco. No entanto, o cliente de testes do Flask (`app.test_client()`) abre sessões próprias ao processar cada requisição, e essas sessões participam de transações independentes da transação do teste. Como consequência, o *rollback* da transação do teste não desfaz as alterações feitas pelas requisições HTTP — o isolamento falha.

O `TRUNCATE` resolve o problema porque opera no nível do banco, fora do controle transacional das sessões da aplicação. A limpeza acontece em uma conexão dedicada, com `commit` explícito, independente das sessões abertas pelo cliente de teste. O Algoritmo 6 descreve a função de limpeza em pseudocódigo.

A escolha do `TRUNCATE` tem uma contrapartida: ele é mais lento que o *rollback*, porque envolve I/O real e reinício de sequências. Para a suíte do módulo de solicitações, com poucas dezenas de testes, o custo é irrelevante. Para suítes maiores, o *rollback* transacional seria preferível, desde que o cliente de testes fosse adaptado para participar da transação — o que exigiria reescrever os testes em um estilo diferente. A escolha do `TRUNCATE` privilegia simplicidade e robustez sobre desempenho, o que é adequado ao estágio atual do projeto.

3.5.2.1 Fixtures de Autenticação

Os testes do módulo de solicitações exigem, em sua maioria, um usuário autenticado — solicitante ou avaliador. Sem abstração, cada teste precisaria criar o usuário, realizar a autenticação, obter o token e montar o cabeçalho de autorização, repetindo aproximadamente quinze linhas de código. Para evitar essa duplicação, foram implementadas duas fixtures de autenticação: `auth_specialist` e `auth_administrator`.

A fixture `auth_specialist` cria um usuário do tipo especialista com dados gerados por `UserFake`, autentica-o via `POST /users/login`, extrai o token da resposta e cria os dados pessoais do especialista via `POST /specialists/personal_data`. Retorna o cabeçalho de autorização pronto para ser utilizado pelo cliente de testes. A fixture aceita um payload opcional, permitindo que cada teste personalize os dados do usuário sem duplicar a lógica de autenticação.

Algoritmo 6: Limpeza de tabelas transacionais entre testes (pseudocódigo).

```
TABELAS_TRANSACIONAIS = [
    "specialists",
    "users",
    "solicitations",
    "solicitation_reviews",
    "specialist_snapshots",
    "notifications",
]

FUNÇÃO truncar(app, db):
    COM app.em_contexto():
        COM db.engine.conectar() COMO conexão:
            conexão.executar(
                "TRUNCATE TABLE "
                + juntar(TABELAS_TRANSACIONAIS)
                + " RESTART IDENTITY CASCADE")
            conexão.commit()

FIXTURE session(db, app):    // autouse, escopo de função
    truncar(app, db)
    ENTREGAR db.session
    truncar(app, db)
```

A fixture `auth_administrator` segue o mesmo padrão, mas cria um usuário com perfil de administração (`ProfileEnum.ADMIN`) e tipo de vínculo servidor. Utiliza identificador fixo (99999) para tornar o usuário previsível nos testes e evitar dependência da sequência de IDs. Como a fixture de sessão executa `TRUNCATE ... RESTART IDENTITY CASCADE` antes de cada teste, o usuário é recriado a cada execução; não se trata, portanto, de reaproveitamento entre testes, mas de um identificador estável dentro de cada cenário. O perfil de administração satisfaz o `FeatureEnum.SPECIALIST_REVIEW`, exigido pelas rotas de revisão, conforme apresentado na Seção 3.2.1.

A separação em duas fixtures, em vez de uma parametrizada, decorre das diferenças entre os fluxos de preparação: o especialista precisa de dados pessoais para estar completo no sistema, enquanto o administrador requer um perfil específico para acessar as rotas de revisão. A unificação exigiria condicionais na própria fixture, tornando sua implementação menos legível. Nesse contexto, a pequena duplicação de código entre as duas implementações é preferível à introdução dessa complexidade.

O uso das fixtures reduz o código de preparação dos testes e centraliza a lógica de autenticação, assegurando um fluxo consistente entre os diferentes cenários. Dessa forma, eventuais alterações no processo de autenticação, como mudanças no formato do token, podem ser tratadas em um único ponto.

3.5.3 Testes de integração

Os testes de integração verificaram a comunicação entre os módulos e a correta interação entre backend, banco de dados e interfaces do sistema. Essa camada de validação foi importante para confirmar que as funcionalidades operavam de forma coerente no contexto completo da aplicação.

3.5.4 Processo de Integração Contínua

O processo de integração contínua foi adotado como mecanismo recorrente de validação, permitindo automatizar a execução dos testes e reduzir erros durante o desenvolvimento. Esse fluxo contribui para manter a qualidade do código e garantir que novas alterações não comprometam o funcionamento do sistema. A solução foi implementada com o GitLab CI, usando o Runner interno do Polo de Inovação, o que mantém a execução no ambiente institucional e alinhada à infraestrutura já adotada pelo Laboratório Assert (Seção 2.7).

3.5.4.1 Estrutura do Pipeline

O pipeline foi organizado em dois estágios sequenciais, `lint` e `test`, de modo que as verificações de estilo e tipagem precedem a execução da suíte de testes. Essa separação permite obter feedback rápido sobre problemas de formatação antes de se dispender tempo com a execução completa dos testes. A definição do pipeline foi escrita em `.gitlab-ci.yml`, versionada junto ao código, o que garante que a configuração acompanhe a evolução do projeto e seja reproduzível em outros ambientes.

3.5.4.1.1 Estágio de lint

O job de lint está configurado com `allow_failure: true`, o que o torna não bloqueante para o merge. A decisão é deliberada: as ferramentas de análise estática (`black`, `mypy` e `ruff`) foram adicionadas ao pipeline, mas as verificações efetivas encontram-se comentadas no arquivo de configuração; por ora, executa-se apenas a instalação das dependências. A motivação é evitar que ruído de formatação em código legado impeça a integração de alterações funcionais, sobretudo em um módulo novo convivendo com controladores que não seguem um padrão uniforme. A ativação das verificações como bloqueantes foi registrada como evolução prevista, condicionada à padronização prévia do código legado.

3.5.4.1.2 Estágio de testes

Os jobs de teste herdam de um template comum, `.test_template`, que define o estágio e a preparação do ambiente. Como a suíte depende de PostgreSQL e Redis, o template aguarda a disponibilidade de ambos antes de iniciar os testes: para o banco, utiliza-se `pg_isready` em laço com intervalo de dois segundos; para o Redis, `redis-cli ping` com a mesma lógica.

Essa espera ativa evita a condição de corrida típica de ambientes conteinerizados, em que o job de teste inicia antes de os serviços estarem prontos para aceitar conexões — uma falha intermitente, difícil de diagnosticar e que compromete a confiabilidade do pipeline.

A execução dos testes foi dividida em quatro jobs por módulo, cada um responsável por um diretório de testes:

- `test-geral-module`, para `test/modules/geral`;
- `test-user-module`, para `test/modules/user`;
- `test-specialist-module`, para `test/modules/specialist`;
- `test-solicitation-module`, para `test/modules/solicitation`.

Todos executam o `pytest` com saída verbosa e nível de log `INFO`, o que permite reconstituir a execução em caso de falha. Os jobs de especialista e de solicitação declaram `needs: [test-user-module]`, refletindo a dependência funcional entre esses módulos e o de usuário: como especialistas e solicitações referenciam usuários, uma falha no módulo base invalida a interpretação dos demais. A divisão por módulo foi preferida a um único *job* com toda a suíte por dois motivos: o paralelismo reduz o tempo total de execução, e o detalhamento por módulo torna o diagnóstico mais direto — quando a pipeline falha, o job indica imediatamente qual área foi afetada.

3.5.4.2 Gatilhos e Bloqueio de Merge

O pipeline é executado tanto em *push* quanto em *merge request*. A execução em *push* fornece feedback contínuo ao desenvolvedor durante o trabalho na ramificação; a execução em *merge request* atua como verificação prévia à integração à base principal. Em caso de falha em qualquer job de teste, o merge é bloqueado, impedindo que alterações que quebrem as regras de negócio ou o comportamento existente sejam incorporadas. A única exceção é o job de lint, que, por estar configurado com `allow_failure: true`, não bloqueia o merge enquanto as verificações estiverem desativadas. Essa assimetria é intencional: os testes representam a garantia mínima de correção e, por isso, bloqueiam; o lint, no estágio atual, é informativo.

A adoção do bloqueio no *merge request*, e não apenas no *push*, desloca a verificação para o momento em que ela é efetivamente decisiva: a integração. O desenvolvedor pode iterar livremente em sua ramificação, mas a base principal só recebe alterações que passaram pela suíte completa. Isso concretiza, no fluxo de trabalho, o princípio de que o controle de versão atua sobre o código e a integração contínua atua sobre a sua validade (Seção 2.7).

4 CONSIDERAÇÕES FINAIS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

Referências

BAYER, M. Sqlalchemy. In: BROWN, A.; WILSON, G. (Ed.). **The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks**. Lulu.com, 2012. Disponível em: <https://aosabook.org/en/v2/sqlalchemy.html>. Acesso em: 25 out. 2026. Citado na página 8.

BECK, K.; ANDRES, C. **Extreme Programming Explained: Embrace Change**. 2. ed. Boston, MA: Addison-Wesley Professional, 2004. 224 p. ISBN 978-0321278654. Citado na página 7.

COHN, M. **Succeeding with Agile: Software Development Using Scrum**. Boston: Addison-Wesley Professional, 2009. Citado na página 7.

DOCKER. **Docker Documentation**. [S.l.], 2024. Disponível em: <https://docs.docker.com/>. Acesso em: 26 out. 2026. Citado na página 16.

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [S.l.]: Addison-Wesley, 2003. Citado 2 vezes nas páginas 10 e 11.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, Irvine, CA, USA, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Citado na página 5.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/info/rfc9110>. Acesso em: 25 out. 2026. Citado 2 vezes nas páginas 5 e 6.

FOWLER, M. **Audit Log**. 2004. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/AuditLog.html>. Acesso em: 25 out. 2026. Citado 2 vezes nas páginas 6 e 9.

FOWLER, M. **Event Sourcing**. 2005. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/EventSourcing.html>. Acesso em: 25 out. 2026. Citado na página 10.

FOWLER, M. **Padrões de Arquitetura de Aplicações Corporativas**. Porto Alegre: Bookman, 2006. Obra original publicada em 2002. Citado na página 6.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, Massachusetts: Addison-Wesley, 1994. Citado 2 vezes nas páginas 5 e 10.

Git. **Git: fast-version-control**. 2026. Disponível em: <https://git-scm.com/>. Acesso em: 27 out. 2026. Citado na página 16.

GitLab. **GitLab Documentation**. 2026. <https://docs.gitlab.com/>. Acesso em: 25 out. 2026. Citado na página 16.

GLAS, B. et al. **OWASP Top 10:2025**. [S.l.], 2025. Acessado em: 2 de novembro de 2026. Disponível em: <https://owasp.org/Top10/2025/>. Citado na página 6.

HOLGER KREKEL AND PYTEST DEVELOPERS. **pytest: helps you write better programs**. [S.l.], 2024. Disponível em: <https://docs.pytest.org/>. Acesso em: 25 out. 2026. Citado na página 15.

KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 10.

MARSHMALLOW DEVELOPERS. **marshmallow: simplified object serialization**. [S.l.], 2024. Disponível em: <https://marshmallow.readthedocs.io/>. Acesso em: 25 out. 2026. Citado na página 9.

NUXT. **Nuxt: The Intuitive Web Framework**. [S.l.], 2024. Disponível em: <https://nuxt.com/docs/getting-started/introduction>. Acesso em: 25 out. 2026. Citado na página 12.

OpenProject. **OpenProject Documentation**. 2026. <https://www.openproject.org/docs/>. Acesso em: 25 out. 2026. Citado 2 vezes nas páginas 3 e 7.

PALLETS. **Flask Documentation**. [S.l.], 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 25 out. 2026. Citado na página 8.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. [S.l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 out. 2026. Citado na página 8.

SANDHU, R. S. et al. Role-based access control models. **Computer**, IEEE, v. 29, n. 2, p. 38–47, 1996. Citado na página 14.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado 2 vezes nas páginas 10 e 16.

TANENBAUM, A. S.; van Steen, M. **Sistemas Distribuídos: Princípios e Paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007. Citado na página 5.

VUE.JS. **Vue.js: The Progressive JavaScript Framework**. [S.l.], 2024. Disponível em: <https://vuejs.org/guide/introduction.html>. Acesso em: 25 out. 2026. Citado na página 11.

Apêndices

APÊNDICE A – Nome do apêndice

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros apêndices para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do apêndice.

Não é aconselhável colocar tudo que é complementar em um único apêndice. Organize os apêndices de modo que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho.

APÊNDICE B – Nome do outro apêndice

conteúdo do novo apêndice

Anexos

ANEXO A – Nome do anexo

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros anexos para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do anexo.

Organize seus anexos de modo a que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho. É para ele que você escreve.

ANEXO B – Nome do outro anexo

conteúdo do outro anexo